



Kĩ thuật lập trình - Hệ thống điểm danh

Kĩ thuật lập trình (Trường Đại học Bách khoa Hà Nội)



messages.pdf_cover_qr_code_label

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN – TIN



BÁO CÁO CUỐI KỲ HỌC PHẦN
KỸ THUẬT LẬP TRÌNH
ĐỀ TÀI:
HỆ THỐNG QUẢN LÝ ĐIỂM DANH LỚP HỌC

Giảng viên hướng dẫn: Vũ Thành Nam

Sinh viên thực hiện: Trần Đăng Ninh

Mã số sinh viên: 20237470

Mã lớp học: 158241

Nhóm: 216

Hà Nội, tháng 6 năm 2025

LỜI MỞ ĐẦU

Lời đầu tiên, em xin gửi lời cảm ơn sâu sắc đến thầy Vũ Thành Nam. Sự giảng dạy, hỗ trợ và tạo điều kiện của thầy đã giúp em hoàn thành tốt nhất báo cáo này. Chúc thầy có thật nhiều sức khỏe để tiếp tục công tác giảng dạy và nghiên cứu khoa học trong thời gian tới!

Trong thời đại hiện nay, công nghệ thông tin đóng vai trò then chốt trong hầu hết mọi lĩnh vực của đời sống. Để xây dựng nên các hệ thống phần mềm hiệu quả, bền vững và dễ bảo trì, kỹ thuật lập trình không chỉ là một kỹ năng mà còn là một nền tảng tư duy quan trọng đối với mỗi lập trình viên.

Kỹ thuật lập trình giúp người học hình thành cách tiếp cận bài toán một cách logic, tối ưu hóa giải pháp, tổ chức mã nguồn khoa học và dễ mở rộng. Không chỉ đơn thuần là viết mã cho chạy được, kỹ thuật lập trình còn hướng tới viết mã đúng, mã sạch và mã tốt — những yếu tố cốt lõi làm nên sự thành công của các dự án phần mềm thực tiễn.

Với mục đích nghiên cứu và ứng dụng kỹ thuật lập trình vào thực tiễn, em xin được giới thiệu đến thầy bài báo cáo cuối kỳ học phần "Kỹ thuật lập trình" (MI3310) với đề tài "Hệ thống quản lý điểm danh lớp học".

Đề tài "Hệ thống quản lý điểm danh lớp học" được thực hiện nhằm xây dựng một phần mềm hỗ trợ giáo viên và nhà trường trong việc theo dõi, cập nhật, truy vấn và báo cáo thông tin điểm danh của sinh viên theo lớp học, theo ngày và theo trạng thái tham dự. Hệ thống không chỉ giúp giảm tải công việc thủ công mà còn tạo tiền đề cho việc tích hợp với các hệ thống quản lý học vụ trong tương lai. Em hy vọng rằng báo cáo này sẽ phản ánh quá trình nghiên cứu, thiết kế, cài đặt và kiểm thử dự án; cung cấp góc nhìn về cách em ứng dụng những kiến thức đã học được trong học phần vào một dự án mang tính thực tiễn.

Hà Nội, ngày 9 tháng 6 năm 2025

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU VỀ ĐỀ TÀI.....	4
1.1 Mô tả chủ đề và quá trình thực hiện.....	4
CHƯƠNG 2. CẤU TRÚC CHƯƠNG TRÌNH.....	4
2.1 Mô tả về cấu trúc chương trình.....	4
2.2 Xây dựng các module thực hiện chức năng.....	5
2.2.1 Module 1: Định nghĩa các đối tượng.....	5
2.2.2 Module 2: Quản lý dữ liệu.....	6
2.2.3 Module 3: Thực hiện điểm danh và cập nhật.....	7
2.2.4 Module 4: Hệ thống tìm kiếm & truy vấn.....	7
2.2.5 Module 5: Hiển thị kết quả tìm kiếm & truy vấn.....	8
2.2.6 Module 6: Điều phối chương trình.....	9
2.2.7 Hàm main.....	10
CHƯƠNG 3. THIẾT KẾ CHƯƠNG TRÌNH.....	12
3.1 Kỹ thuật thiết kế cấu trúc chương trình.....	12
3.1.1 Các mô thức lập trình được sử dụng.....	12
3.1.2 Các kỹ thuật quản lý và xử lý dữ liệu.....	15
3.1.3 Kỹ thuật dùng hàm và tham số.....	16
3.2 Kỹ thuật đặt tên và trình bày mã trong chương trình.....	17
3.2.1 Kỹ thuật đặt tên trong chương trình.....	17
3.2.2 Trình bày mã trong chương trình.....	18
CHƯƠNG 4. GỠ LỖI VÀ KIỂM THỬ.....	20
4.1 Xác định & Gỡ lỗi.....	20
4.2 Kiểm thử.....	21
CHƯƠNG 5. CÁC TÌNH HUỐNG KIỂM THỬ.....	23
5.1 Kiểm thử menu chính.....	23
5.1.1 Test case 1: Hiển thị menu chính.....	23
5.1.2 Test case 2: Lựa chọn không hợp lệ (vượt khoảng giá trị menu)..	23
5.1.3 Test case 3: Lựa chọn đầu vào không phải số.....	23
5.1.4 Test case 4: Lựa chọn thoát chương trình.....	24
5.2 Kiểm thử chức năng 1: Điểm danh cho lớp học.....	24
5.2.1 Test case 5: Điểm danh thành công cho lớp học có sinh viên.....	24

5.2.2	Test case 6: Điểm danh cho lớp không tồn tại	26
5.2.3	Test case 7: Điểm danh với mã lớp để trống	26
5.2.4	Test case 8: Điểm danh với đầu vào ngày sai định dạng.....	27
5.2.5	Test case 9: Điểm danh với lớp không có sinh viên.....	27
5.2.6	Test case 10: Nhập trạng thái không hợp lệ cho sinh viên	28
5.2.7	Test case 11: Điểm danh lại cho một buổi điểm danh (ghi đè)	29
5.3	Kiểm thử chức năng 2: Cập nhật trạng thái điểm danh.....	30
5.3.1	Test case 12: Cập nhật điểm danh thành công	30
5.3.2	Test case 13: Cập nhật bản ghi điểm danh không tồn tại	30
5.3.3	Test case 14: Cập nhật với trạng thái mới không hợp lệ	31
5.3.4	Test case 15: Cập nhật trạng thái giống trạng thái cũ.....	31
5.3.5	Test case 16: Để trống các trường input bắt buộc	32
5.4	Kiểm thử chức năng 3: Tìm kiếm sinh viên	33
5.4.1	Test case 17: Tìm kiếm chính xác theo mã sinh viên	33
5.4.2	Test case 18: Tìm kiếm mã sinh viên sai quy tắc nhập.....	33
5.4.3	Test case 19: Tìm kiếm sinh viên theo tên đầy đủ	34
5.4.4	Test case 20: Tìm kiếm sinh viên theo một phần tên.....	34
5.4.5	Test case 21: Tìm kiếm sinh viên với tên hoặc mã sai định dạng..	35
5.4.6	Test case 22: Tìm kiếm sinh viên với input rỗng.....	35
5.5	Kiểm thử chức năng 4: Tìm kiếm lớp học.....	35
5.5.1	Test case 23: Tìm kiếm chính xác theo mã lớp học.....	36
5.5.2	Test case 24: Tìm kiếm sai mã lớp học	36
5.5.3	Test case 25: Tìm kiếm lớp học theo tên lớp.....	36
5.5.4	Test case 26: Tìm kiếm theo một phần tên lớp học.....	37
5.5.5	Test case 27: Tìm kiếm lớp học không có trong danh sách	37
5.5.6	Test case 28: Tìm kiếm lớp học với input rỗng.....	38
5.6	Kiểm thử chức năng 5: Xem lịch sử điểm danh của sinh viên	38
5.6.1	Test case 29: Xem lịch sử của SV, không lọc lớp	38
5.6.2	Test case 30: Xem lịch sử của SV, có lọc lớp hợp lệ	38
5.6.3	Test case 31: Xem lịch sử của SV; có lọc lớp, chưa điểm danh	39
5.6.4	Test case 32: Xem lịch sử của SV không tồn tại.....	39
5.6.5	Test case 33: Xem lịch sử của SV, nhập mã lớp không tồn tại	40
5.7	Kiểm thử chức năng 6: Xem báo cáo điểm danh của lớp học.....	40
5.7.1	Test case 34: Xem báo cáo của lớp học, không lọc ngày	40
5.7.2	Test case 35: Xem báo cáo của lớp, có lọc ngày hợp lệ.....	41

5.7.3	Test case 36: Xem báo cáo của lớp, lọc ngày không có dữ liệu.....	42
5.7.4	Test case 37: Xem báo cáo của lớp, lọc ngày sai định dạng	43
5.7.5	Test case 38: Xem báo cáo cho lớp không tồn tại.....	43
CHƯƠNG 6. TỔNG KẾT		45
6.1	Ưu điểm	45
6.2	Nhược điểm và hướng mở rộng.....	45
CHƯƠNG 7. PHỤ LỤC		46
7.1	Mã nguồn hàm main	46
7.2	Đặc tả các chương trình con	46
TÀI LIỆU THAM KHẢO.....		49

CHƯƠNG 1. GIỚI THIỆU VỀ ĐỀ TÀI

1.1 Mô tả chủ đề và quá trình thực hiện

Chủ đề của bài báo cáo của em là “Xây dựng hệ thống quản lý điểm danh lớp học”. Cụ thể hơn, nhiệm vụ của em là xây dựng một hệ thống cho phép ghi nhận điểm danh theo ngày cho từng sinh viên (mã sinh viên, họ tên) cho các lớp học và cho phép tìm kiếm theo ngày của lớp hoặc theo mã sinh viên.

Hệ thống điểm danh em xây dựng sẽ nhận các đầu vào sau:

- Danh sách sinh viên sẽ được phân bổ vào các lớp học trong hệ thống.
- Danh sách các lớp học trong hệ thống.

Hệ thống được xây dựng với sáu chức năng chính:

- Cho phép thực hiện việc điểm danh theo ngày, theo lớp học với từng sinh viên.
- Cho phép sửa đổi (cập nhật) trạng thái điểm danh của sinh viên sau khi thực hiện điểm danh.
- Cho phép tìm kiếm các lớp học có trong hệ thống hay không.
- Cho phép tìm kiếm các sinh viên có trong hệ thống hay không.
- Cho phép xem lịch sử điểm danh của từng sinh viên (có bộ lọc theo ngày và theo lớp học).
- Cho phép xem báo cáo điểm danh của các lớp học (có bộ lọc theo ngày).

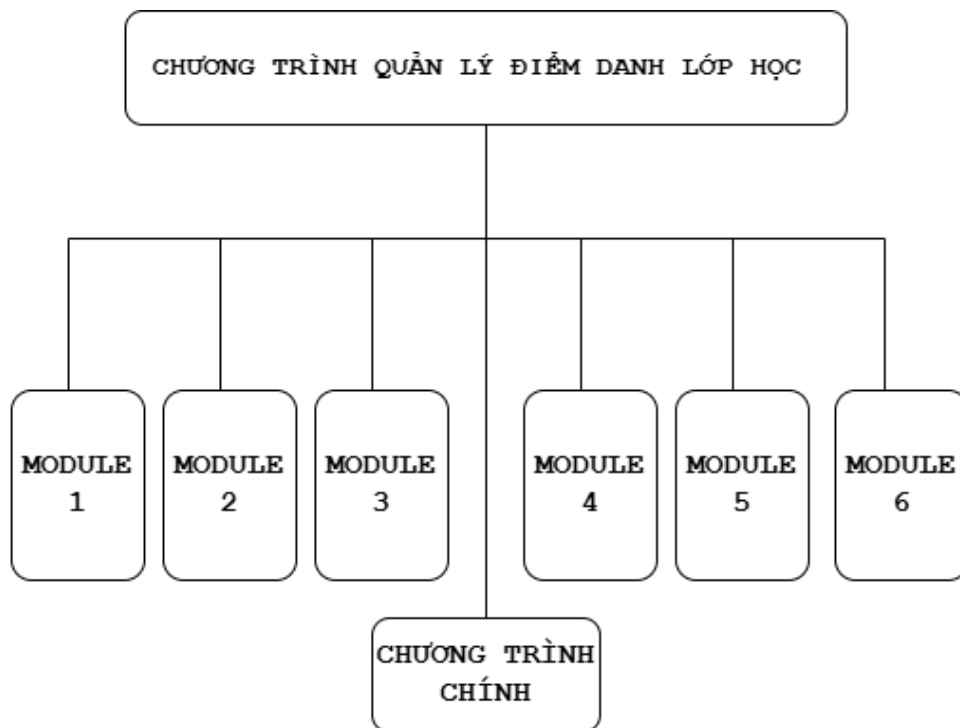
Thực hiện chủ đề này, bước đầu tiên sau khi đọc các yêu cầu, em đã phác thảo trước cấu trúc chương trình ra giấy, nếu để tất cả chương trình trong cùng một file thì vừa quá dài vừa bất tiện cho việc theo dõi logic nên em đã tách ra thành sáu module thực hiện các công việc riêng biệt (đảm bảo phong cách lập trình tốt – module hóa chương trình).

Trong thư mục của toàn bộ dự án em tạo thêm một thư mục tên là “data”; thư mục này chứa hai file dữ liệu đầu vào và một file ghi lại dữ liệu đầu ra – đó là các bản ghi điểm danh cho từng sinh viên theo ngày và theo lớp – sau khi thực hiện chức năng điểm danh hoặc cập nhật điểm danh.

CHƯƠNG 2. CẤU TRÚC CHƯƠNG TRÌNH

2.1 Mô tả về cấu trúc chương trình

Dưới đây là sơ đồ mô tả về cấu trúc chương trình:



Bên cạnh đó, dưới đây là sơ đồ cấu trúc thư mục chứa dự án:

Dự án môn KTLT/

```

├── __pycache__      #Thư mục chứa các file .pyc do Python sinh ra
├── data              #Thư mục chứa dữ liệu (JSON)
│   ├── students.json
│   ├── classes.json
│   └── attendance_records.json
├── models.py         # Module 1
├── data_manager.py   # Module 2
├── attendance_taking_system.py # Module 3
├── search_service.py # Module 4
├── display_search_service.py # Module 5
├── program_controller.py # Module 6
└── main.py           # Chương trình chính
  
```

2.2 Xây dựng các module thực hiện chức năng

2.2.1 Module 1: Định nghĩa các đối tượng

Trong module 1, em định nghĩa ba đối tượng chính dùng trong chương trình theo nguyên lý lập trình hướng đối tượng bằng cách khởi tạo ba class: Student, Class và AttendanceRecord.

Các thông tin được lưu trữ trong class Student:

- Mã sinh viên (student_id): được lưu trữ bằng kiểu dữ liệu string.

- Tên sinh viên (full_name): được lưu trữ bằng kiểu dữ liệu string.

Các thông tin được lưu trữ trong class Class:

- Mã lớp học (class_id): được lưu trữ bằng kiểu dữ liệu string.
- Tên lớp học (class_name): được lưu trữ bằng kiểu dữ liệu string.

Các thông tin được lưu trữ trong class AttendanceRecord:

- Mã sinh viên (student_id): được lưu trữ bằng kiểu dữ liệu string.
- Mã lớp học (class_id): được lưu trữ bằng kiểu dữ liệu string.
- Ngày điểm danh (attendance_date): được lưu trữ bằng kiểu dữ liệu string, dạng YYYY– MM – DD.
- Trạng thái điểm danh (status): được lưu trữ bằng kiểu dữ liệu string.

Tất cả các class đều có các phương thức `_init_` (khởi tạo), `_str_` (trả về chuỗi dễ đọc, thân thiện với người dùng), `_repr_` (trả về chuỗi kỹ thuật dành cho debug) và một phương thức đặc biệt là `change_object_to_dict` (chuyển đổi tượng thành kiểu dictionary – dùng trong module 2 quản lý dữ liệu).

Bên cạnh đó, riêng với class Class; em đã khởi tạo thêm phương thức `get_student_ids`: Trả về các mã sinh viên trong lớp (dùng trong module 3 thực hiện điểm danh).

2.2.2 Module 2: Quản lý dữ liệu

Module 2 thực hiện công việc quản lý dữ liệu đầu vào và đầu ra lấy từ các file JSON.

Các công việc được thực hiện trong module 2:

1. Xác định đường dẫn tới các file dữ liệu và lưu vào biến toàn cục.
2. Kiểm tra thư mục dữ liệu có tồn tại hay không, nếu không thì tạo mới thư mục data/
3. Thực hiện thao tác tải dữ liệu:
 - Mở và đọc file JSON tương ứng với đối tượng được xử lý, nếu file không tồn tại thì trả về danh sách rỗng.
 - Chuyển đổi dữ liệu file JSON (dạng text) thành danh sách các dictionary.
 - Tạo đối tượng Python tương ứng từ danh sách các dictionary và trả về các đối tượng được tạo.
4. Thực hiện thao tác lưu dữ liệu:
 - Chuyển đối tượng truyền vào thành các dictionary thông qua các phương thức chuyển đổi của từng loại đối tượng trong module 1.
 - Tạo danh sách mới chứa tất cả các dictionary chuyển đổi.
 - Ghi dữ liệu vào file JSON.

2.2.3 Module 3: Thực hiện điểm danh và cập nhật

Module 3 thực hiện hai chức năng cốt lõi của hệ thống là điểm danh và cập nhật các bản ghi điểm danh.

Các công việc chính thực hiện trong module này có thể kể ra như:

1. Tạo các biến toàn cục có chức năng như cache dữ liệu. Tải và nạp các dữ liệu trong file JSON vào các cache.
2. Thực hiện chức năng điểm danh:
 - Tải dữ liệu nếu cần.
 - Tìm lớp học phù hợp với class_id người dùng nhập.
 - Hiển thị các thông tin điểm danh: tên lớp, ngày điểm danh, các trạng thái điểm danh hợp lệ.
 - Lấy danh sách sinh viên có trong lớp và điểm danh.
 - Thêm các bản ghi điểm danh mới (nếu trước đó đã tồn tại các bản ghi cũ cho cùng ngày thì xóa và ghi đè), cập nhật lại cache và lưu các bản ghi vào file attendance_record.json.
3. Thực hiện chức năng cập nhật điểm danh:
 - Tải dữ liệu nếu cần.
 - Kiểm tra hợp lệ và chuẩn hóa các trạng thái mới theo đúng định dạng.
 - Cập nhật trạng thái mới và lưu.

2.2.4 Module 4: Hệ thống tìm kiếm & truy vấn

Module này thực hiện các chức năng hỗ trợ mà chủ đề yêu cầu như cho phép tìm kiếm báo cáo điểm danh theo ngày của lớp hoặc tìm kiếm báo cáo điểm danh theo mã sinh viên (xem lịch sử điểm danh của sinh viên).

Các công việc chính thực hiện trong module này có thể kể ra như:

1. Đảm bảo dữ liệu đã được tải vào cache, nếu cache trống thì sẽ gọi các hàm tải dữ liệu trong module 2 để nạp vào cache.
2. Nếu người dùng thực hiện chức năng “Tìm kiếm sinh viên”:
 - Chuẩn hóa lệnh tìm kiếm (query) nhập từ bàn phím (loại bỏ khoảng trắng thừa ở đầu và cuối chuỗi nhập; đối với mã sinh viên thì cần nhập chính xác chữ hoa/chữ thường, với tên sinh viên thì không cần).
 - Lặp qua từng đối tượng student trong cache.
 - So sánh đầu vào với dữ liệu trong cache, lọc đối tượng và trả về kết quả.
3. Nếu người dùng thực hiện chức năng “Tìm kiếm lớp học”:
 - Chuẩn hóa lệnh tìm kiếm (query) nhập từ bàn phím (loại bỏ khoảng trắng thừa ở đầu và cuối chuỗi nhập; đối với mã lớp học thì cần nhập chính xác chữ hoa/chữ thường, với tên lớp học thì không cần). Trong hàm thực hiện chức năng này, em đã đổi tên các biến liên quan đến đối tượng Class thành định

dạng `school_class...` để tránh nhầm lẫn với từ khóa `class` của Python.

- Lặp qua từng đối tượng `student` trong `cache`.
 - So sánh đầu vào với dữ liệu trong `cache`, lọc đối tượng và trả về kết quả.
4. Nếu người dùng thực hiện chức năng “Xem lịch sử điểm danh của sinh viên”:
- Thực hiện chuẩn hóa lệnh tìm kiếm nhập từ bàn phím.
 - Lặp qua `cache` lưu các bản ghi điểm danh.
 - Lọc theo sinh viên (so sánh đầu vào mã sinh viên với các dữ liệu trong các bản ghi `cache`) và có thể lọc theo lớp (bỏ trống để hiển thị toàn bộ dữ liệu liên quan đến sinh viên trong tất cả các lớp).
 - Sắp xếp kết quả theo ngày và mã lớp, sau đó in ra màn hình.
5. Nếu người dùng thực hiện chức năng “Xem báo cáo điểm danh của lớp học”:
- Thực hiện chuẩn hóa lệnh tìm kiếm nhập từ bàn phím.
 - Xử lý bộ lọc theo ngày: chuyển đổi chuỗi đầu vào (ngày điểm danh) nhập từ bàn phím thành đối tượng `date`, nếu quá trình chuyển đổi thất bại thì truy vấn sẽ không lọc theo ngày.
 - Lặp qua `cache` lưu các bản ghi điểm danh.
 - Lọc theo lớp học (so sánh đầu vào mã lớp học với các dữ liệu trong bản ghi `cache`) và có thể lọc theo ngày (nếu quá trình chuyển đổi ở bước trên thành công)
 - Sắp xếp kết quả theo ngày điểm danh và mã sinh viên, sau đó in ra màn hình

2.2.5 Module 5: Hiển thị kết quả tìm kiếm & truy vấn

Module 5 thực hiện hiển thị kết quả tìm kiếm & truy vấn thực hiện trong module 4 ra màn hình. Ban đầu, em dự định gộp chung hai module này thành một nhưng để thuận tiện cho quá trình sửa đổi và góp ý của thầy, đồng thời đảm bảo tính module hóa của chương trình nên em đã tách biệt module này ra.

Các công việc chính thực hiện trong module này có thể kể ra:

1. Nếu người dùng chọn xem báo cáo điểm danh của sinh viên, hệ thống sẽ gọi hàm `display_attendance_records` để hiển thị các bản ghi điểm danh:
 - Đầu tiên, chương trình sẽ kiểm tra dữ liệu các bản ghi điểm danh có tồn tại không (trong file `attendance_record.json`).
 - Nếu tồn tại, sau đó chương trình xác định tiêu đề hiển thị ra màn hình.
 - Bước tiếp theo, chương trình chuẩn bị dữ liệu để tra cứu bằng cách tạo hai biến kiểu `dictionary` là `student_map` và `class_map`.

- In tiêu đề cột (header) của bảng kết quả.
 - Cuối cùng, chương trình lặp qua từng đối tượng record trong danh sách records để lấy dữ liệu xuất ra màn hình.
2. Nếu người dùng chọn xem báo cáo điểm danh của lớp học, hệ thống sẽ gọi hàm `display_class_attendance_details`:
- Đầu tiên, chương trình sẽ lấy thông tin lớp (bằng cách dùng phương thức `get_class_by_id` nằm trong module 3 thực hiện điểm danh).
 - Sau đó, chương trình xác định tiêu đề hiển thị ra màn hình.
 - Sau khi in tiêu đề, nếu không có lớp học nào tồn tại để lấy thông tin thì sẽ in thông báo lỗi.
 - Trường hợp khác, nếu không tồn tại bản ghi điểm danh nào – có thể là chưa điểm danh hoặc lớp học có tồn tại nhưng không có sinh viên thì sẽ in thông báo không có dữ liệu.
 - In tiêu đề các cột của bảng kết quả.
 - Cuối cùng, chương trình lặp qua danh sách các dictionary để lấy dữ liệu và xuất ra màn hình.
3. Nếu người dùng muốn tìm kiếm xem sinh viên nào đó có tồn tại hay không, hệ thống gọi hàm `display_found_students`:
- Đầu tiên, chương trình kiểm tra danh sách students, nếu không có sinh viên thỏa mãn đầu vào thì báo không tìm thấy và kết thúc.
 - Sau đó sẽ in tiêu đề của kết quả được hiển thị.
 - Cuối cùng, chương trình lặp qua các đối tượng trong danh sách students, tìm đối tượng thỏa mãn đầu vào để lấy và xuất ra màn hình.
4. Nếu người dùng muốn tìm kiếm lớp học nào đó có tồn tại hay không, hệ thống sẽ gọi hàm `display_found_classes`:
- Đầu tiên, chương trình kiểm tra danh sách classes, nếu không có lớp học thỏa mãn đầu vào thì báo không tìm thấy và kết thúc.
 - Sau đó sẽ in tiêu đề của kết quả được hiển thị.
 - Cuối cùng, chương trình lặp qua các đối tượng trong danh sách classes, tìm đối tượng thỏa mãn đầu vào để lấy và xuất ra màn hình.

2.2.6 Module 6: Điều phối chương trình

Module này thực hiện điều phối chính, nhận lệnh từ chương trình chính (main.py). Trong module này, em đã tạo thêm một đối tượng (object) mới là `AppController`.

Em đã thiết kế module điều phối này theo mô hình ba thành phần Model – View – Controller để tách biệt logic và giao diện người dùng, góp phần đảm bảo nguyên lý lập trình hướng sự kiện và hỗ trợ nâng cấp từ giao diện dòng lệnh (console) lên giao diện đồ họa (GUI) sau này.

Các công việc chính mà module này thực hiện như sau:

1. Khởi tạo đối tượng mới tên là ApplicationController:
 - Model – Yêu cầu module 3 nạp toàn bộ dữ liệu (sinh viên, lớp học, bản ghi điểm danh) từ các file JSON vào cache và in thông báo chi tiết về tình trạng tải dữ liệu.
2. Hiển thị menu và nhận lựa chọn:
 - View – In menu lựa chọn ra màn hình.
 - View – Nhận lựa chọn từ người dùng.
3. Xử lý lựa chọn:
 - Controller – Sử dụng cấu trúc if/elif/else và một list các dictionary (action_map) để ánh xạ lựa chọn của người dùng tới hành động cần thiết thực hiện.
 - Controller – Gọi phương thức xử lý tương ứng với lựa chọn.
4. Thực thi một chức năng cụ thể được gọi sau khi lựa chọn:
 - View – Tương tác ban đầu với người dùng: sử dụng input() để thu thập các thông tin cần thiết nhập từ bàn phím.
 - Controller – Làm sạch & chuẩn hóa input.
 - Controller (gọi Model) – Thực hiện nghiệp vụ chính (gọi các phương thức tích hợp trong các module được thiết kế riêng cho nghiệp vụ).
 - Controller (gọi View) – Hiển thị kết quả/phản hồi.

2.2.7 Hàm main

Bởi vì đã có module 6 – program_controller.py thực hiện điều phối chính, đảm nhận phần lớn logic điều khiển nên vai trò của hàm này chỉ là tạo ra đối tượng ApplicationController, chạy vòng lặp chính để nhận lựa chọn của người dùng và xử lý.

Để nắm rõ cách chương trình hoạt động, ta có thể điểm qua các công việc thực hiện trong hàm này:

1. Khối if `__name__ == "__main__"` được thực thi, đây là điểm vào tiêu chuẩn cho script Python – đảm bảo mã nguồn bên trong chỉ được thực thi khi khối này được gọi.
2. Gọi thành phần Controller để tạo đối tượng ApplicationController.
3. Khởi tạo biến điều khiển vòng lặp.
4. Bắt đầu vòng lặp với điều kiện miễn là biến running là True:
 - Gọi thành phần Controller để hiển thị menu và lấy lựa chọn từ người dùng.
 - Gọi thành phần Controller để xử lý lựa chọn, thực hiện nghiệp vụ tương ứng và trả về kết quả.
 - Nếu chưa thoát ngay, chương trình chờ người dùng tiếp tục.
5. Kết thúc chương trình.

CHƯƠNG 3. THIẾT KẾ CHƯƠNG TRÌNH

3.1 Kỹ thuật thiết kế cấu trúc chương trình

Chương trình hệ thống quản lý điểm danh lớp học được thiết kế theo cơ chế top – down: từ cấp độ cao nhất của hệ thống dần phân chia thành các thành phần nhỏ hơn để dễ quản lý và phát triển.

Bên cạnh đó, em cũng chú ý thiết kế theo hướng module hóa (mỗi module là một file) nhằm đảm bảo cho chương trình có cấu trúc tốt và tách biệt trách nhiệm giữa các phần chương trình khác nhau.

Ưu điểm của thiết kế hướng module hóa:

- Dễ hiểu: Việc tách chương trình thành các module khác nhau giúp người đọc biết và nắm rõ các phần khác nhau của chương trình và công việc chúng đảm nhận.
- Dễ bảo trì, dễ mở rộng: Khi em cần sửa đổi hay cải tiến một chức năng nhất định, việc cần làm chỉ là tìm module tương ứng thực hiện chức năng đó, khoanh vùng vị trí cần xử lý mà ít gây ảnh hưởng đến các phần khác của chương trình (các module khác).
- Dễ kiểm thử: Em có thể kiểm thử từng module riêng biệt, sau đó kiểm thử toàn bộ chương trình (kiểm thử file main.py).
- Không lặp lại code: Với các hàm tiện ích chuyên dùng để thực hiện các công việc khác nhau; việc viết lại cùng dòng lệnh cho hai công việc giống nhau là không cần thiết.

3.1.1 Các mô thức lập trình được sử dụng

Các mô thức lập trình mà em đã sử dụng trong chương trình như:

1. Lập trình mệnh lệnh

Hầu hết các file module trong chương trình của em đều viết theo mô thức lập trình mệnh lệnh. Đặc điểm nổi bật

- Các lệnh được thực thi theo thứ tự được viết của chúng: từ trái sang phải, từ trên xuống dưới kèm theo chú thích trên mỗi khối logic.
- Sử dụng các cấu trúc logic như vòng lặp, câu lệnh điều kiện và lời gọi hàm để điều khiển luồng thực thi.
- Sử dụng biến để lưu trữ trạng thái.

Ví dụ:

```
def handle_update_attendance(self):
    # VIEW: Lấy input
    student_id = input("Nhập mã sinh viên: ").strip()
    class_id = input("Nhập mã lớp học: ").strip()

    # CONTROLLER: Gọi Model để kiểm tra
    student_obj = ats.get_student_by_id(student_id)
    if not student_obj:
        print(f"Sinh viên với mã '{student_id}' không tồn tại.") # VIEW
        return
    class_obj = ats.get_class_by_id(class_id)
    if not class_obj:
        print(f"Lớp học với mã '{class_id}' không tồn tại.") # VIEW
        return

    # VIEW: Lấy input ngày và trạng thái mới
    attendance_date_str = ""
    while True:
        attendance_date_str = input("Nhập ngày điểm danh đã ghi (YYYY-MM-DD): ").strip()
        if not attendance_date_str:
            print("Ngày điểm danh không được để trống.")
            continue
        try:
            date.fromisoformat(attendance_date_str)
            break
        except ValueError:
            print("Định dạng ngày không hợp lệ. Vui lòng nhập lại theo YYYY-MM-DD.")
```

2. Lập trình phòng ngừa

Chương trình được viết theo mô thức lập trình phòng ngừa. Dưới đây là một số đặc điểm của chương trình thể hiện điều đó:

- Có thao tác kiểm tra đầu vào. Ví dụ như trong module 6 program_controller.py, phương thức này chạy lệnh điều kiện if kiểm tra input được nhập từ người dùng.

```
def handle_choice_3(self):
    """
    Đầu vào: Self - tham chiếu đối tượng (ngầm định).
    Đầu ra: Không có
    Phương thức này tìm kiếm sinh viên khi người dùng nhập input là "3"
    """
    query = input("Nhập tên hoặc mã sinh viên cần tìm: ").strip() # VIEW
    if not query: # CONTROLLER
        print("Nội dung tìm kiếm không được để trống.") # VIEW
        return
    students = ss.search_student_by_name_or_id(query) # CONTROLLER -> MODEL
    dsr.display_found_students(students) # CONTROLLER -> VIEW
```

- Có thao tác xử lý lỗi. Ví dụ như trong module 2 data_manager.py có sử dụng khối try – except để bắt lỗi tiềm ẩn và in thông báo lỗi ra màn hình:

```
except FileNotFoundError:
    print(f"Lỗi FileNotFoundError khi truy cập file: {filepath}")
    return []
except json.JSONDecodeError:
    # In thông báo lỗi ra đường dẫn file để biết file nào bị lỗi
    print(f"Lỗi: Dữ liệu JSON không hợp lệ trong file: {filepath}")
    return []
except IOError:
    print(f"Lỗi khi đọc file: {filepath}")
    return []
except Exception as e:
    print(f"Lỗi không xác định khi đọc file {filepath}: {e}")
    return []
```

- Có thao tác xử lý giá trị rỗng. Ví dụ như trong module 3 attendance_taking_system.py:

```
while True:
    try:
        # Nhập trạng thái điểm danh
        status_input = input(f" - {student.full_name} ({student.student_id}): ").strip()
        resolved_status = ""

        # Nếu người dùng không nhập thì coi như "Có mặt"
        if not status_input:
            resolved_status = "Có mặt"
```

- Có sử dụng Optional[Type] cho các tham số hoặc giá trị trả về có thể là None.

3. Lập trình hướng đối tượng (OOP):

Đặc điểm của chương trình cho thấy được viết theo mô thức lập trình hướng đối tượng:

- Thiết kế các lớp đối tượng mô tả các thực thể: Sinh viên, Lớp học, Bản ghi điểm danh và một đối tượng đặc biệt đóng vai trò điều khiển luồng chương trình.
- Mỗi đối tượng đều có các thuộc tính đi kèm (Do là hệ thống quản lý điểm danh nên em đã bỏ bớt một vài thuộc tính không cần thiết như giới tính (sinh viên), địa chỉ (lớp học)...).
- Thể hiện được các tính chất của mô thức này: tính đóng gói (các hành vi), tính kế thừa, tính trừu tượng và tính đa hình.

4. Lập trình hướng sự kiện (dạng thô sơ)

Vòng lặp chính trong chương trình có thể được coi là một dạng vòng lặp sự kiện thô sơ với các sự kiện chính tương đương với việc người dùng chọn các lựa chọn khác nhau trong menu:


```

def display_main_menu(self):
    """
    Đầu vào: Self - tham chiếu đối tượng (ngầm định).
    Đầu ra: Một chuỗi (str) là lựa chọn do người dùng nhập.
    Phương thức này hiển thị menu các lựa chọn cho người dùng
    và nhận đầu vào là lựa chọn.
    """
    print("\n--- HỆ THỐNG ĐIỂM DANH (MVC) ---")
    print("1. Điểm danh cho lớp học")
    print("2. Cập nhật trạng thái điểm danh")
    print("3. Tìm kiếm sinh viên")
    print("4. Tìm kiếm lớp học")
    print("5. Xem lịch sử điểm danh của sinh viên")
    print("6. Xem báo cáo điểm danh của lớp")
    print("0. Thoát")
    return input("Nhập lựa chọn của bạn: ")

def handle_choices(self, choice: str) -> bool:
    """
    Đầu vào:
    + Self - tham chiếu đối tượng (ngầm định)
    + choice (kiểu str): Lựa chọn đầu vào từ menu chính.
    Đầu ra: True để tiếp tục vòng lặp chương trình, False để kết thúc.
    Phương thức nhận đầu vào là lựa chọn của người dùng, sau đó gọi các
    hàm nghiệp vụ riêng cho từng lựa chọn (callback)
    """
    action_map = {
        '1': self.handle_choice_1,
        '2': self.handle_choice_2,
        '3': self.handle_choice_3,
        '4': self.handle_choice_4,
        '5': self.handle_choice_5,
        '6': self.handle_choice_6,
    }

```

Ở đây, phương thức `process_choice` đóng vai trò như một bộ điều phối đáp ứng các “sự kiện” (lựa chọn của người dùng).

Tuy nhiên, việc ứng dụng mô thức lập trình hướng sự kiện trong chương trình này còn thô sơ và thiếu linh hoạt do cấu trúc danh sách các dictionary trong bộ điều phối là cố định và chưa có cơ chế thêm và xử lý các “sự kiện” mới đa dạng hơn.

Đây là điểm hạn chế của hệ thống với giao diện dòng lệnh (Console), mô thức lập trình hướng sự kiện sẽ phù hợp nhất với hệ thống có giao diện đồ họa (GUI).

3.1.2 Các kỹ thuật quản lý và xử lý dữ liệu

Hệ thống điểm danh được thiết kế lấy dữ liệu đầu vào từ các file:

- `students.json`: Chứa danh sách sinh viên, mỗi sinh viên là một đối tượng bao gồm các thuộc tính: Mã sinh viên (`student_id`) và Tên sinh viên (`full_name`).

```

{
    "student_id": "SV130",
    "full_name": "Vũ Anh X"
}

```

- `classes.json`: Chứa danh sách các lớp học, mỗi lớp học là một đối tượng bao gồm các thuộc tính: Mã lớp học (`class_id`), Tên lớp học (`class_name`) và Danh sách sinh viên (`student_ids`).

```
{
  "class_id": "CODETECH01",
  "class_name": "KTLT 01",
  "student_ids": ["SV101", "SV102", "SV103", "SV104", "SV105", "SV106", "SV107", "SV108",
```

- `attendance_records.json`: Chứa danh sách các bản ghi điểm danh; ban đầu thì file này rỗng, là đầu ra để chứa kết quả sau khi người dùng thực hiện các thao tác điểm danh và sửa đổi trạng thái điểm danh nhưng lại là đầu vào cho các thao tác tìm kiếm và truy vấn. Mỗi bản ghi trong file này chứa các thuộc tính:

Một kỹ thuật đặc biệt được sử dụng để quản lý dữ liệu là cache giúp tránh đọc file liên tục. Kỹ thuật này được thực hiện qua việc:

- Khởi tạo các biến toàn cục đóng vai trò như cache.
- Tạo ra các hàm truy cập và lấy dữ liệu nạp vào cache (ví dụ như `get_student_by_id`).

```
# Biến toàn cục để lưu trữ dữ liệu tải từ file (cache)
ALL_STUDENTS: List[Student] = []
ALL_CLASSES: List[Class] = []
ALL_ATTENDANCE_RECORDS: List[AttendanceRecord] = []
```

Ngoài ra, trong chương trình còn có hàm đảm bảo dữ liệu chỉ được tải lên khi cần thiết, tránh I/O ngoài ý muốn.

```
def _ensure_data_loaded():
    """
    Đầu vào: Không có.
    Đầu ra: Không có.
    Hàm này đảm bảo dữ liệu được tải vào cache.
    """
    attendance_taking_system.load_all_data_once()
```

Do chương trình là một hệ thống quản lý điểm danh cho lớp học, không đặt nặng vấn đề quản lý sinh viên nên các thao tác thêm/xóa sinh viên khỏi lớp học em sẽ không tạo các hàm chuyên xử lý các thao tác này mà người dùng có thể trực tiếp sửa đổi trong các file JSON. Tuy nhiên, ta có thể coi đây là một hướng đi mở rộng cho chương trình, vừa tích hợp quản lý sinh viên vừa là hệ thống điểm danh.

3.1.3 Kỹ thuật dùng hàm và tham số

Vì chương trình được thiết kế theo cơ chế top – down nên việc xác định hàm và thủ tục tương tự việc xác định các công việc sẽ được thực hiện trong từng module của chương trình.

Khi đã xác định được công việc của hàm rồi, ta cần phải xác định đầu vào của các hàm để thực hiện chuỗi logic bên trong hàm, cuối cùng trả về kết quả (đầu ra) phù hợp. Tham số giúp tăng tính linh hoạt và tái sử dụng của hàm.

Ví dụ: Hàm này thực hiện việc điểm danh, nhận hai tham số đầu vào:

- Mã lớp học – `class_id` (kiểu string).
- Ngày điểm danh – `attendance_date_str` (kiểu string).

```
def take_attendance(class_id: str, attendance_date_str: str) -> bool:
    """
    Đầu vào:
    - class_id (kiểu str): Mã của lớp học cần điểm danh.
    - attendance_date_str (kiểu str): Ngày điểm danh dưới.
    Đầu ra: True nếu quá trình điểm danh hoàn tất,
            False nếu lớp học không tồn tại.
    Hàm này thực hiện việc điểm danh cho 1 lớp học vào 1 ngày cụ thể.
    """
```

3.2 Kỹ thuật đặt tên và trình bày mã trong chương trình

3.2.1 Kỹ thuật đặt tên trong chương trình

Trong chương trình, em đặt tên các thành phần bằng tiếng Anh, quy tắc đặt tên đều nhất quán và dễ hiểu.

Đối với biến trong chương trình, tên phải đặt là tên có ý nghĩa, gợi nhớ và đặc tả chức năng của nó:

- Biến cục bộ/tham số đầu vào: Tuân thủ quy tắc snake_case.

```
for record in records:
    # Tra cứu thông tin sinh viên
    student_obj = student_map.get(record.student_id)
    student_name = student_obj.full_name if student_obj else "N/A"

    # Tra cứu thông tin lớp học
    class_obj = class_map.get(record.class_id)
    class_name = class_obj.class_name if class_obj else "N/A"
```

- Biến toàn cục (dùng như cache): Tuân thủ quy tắc đặt tên giống biến cục bộ nhưng tất cả đều viết hoa.

```
# Biến toàn cục để lưu trữ dữ liệu tải từ file (cache)
ALL_STUDENTS: List[Student] = []
ALL_CLASSES: List[Class] = []
ALL_ATTENDANCE_RECORDS: List[AttendanceRecord] = []
```

Đối với các phương thức (hàm) thì tên được đặt phải gợi tả chức năng của nó. Để nhất quán, tất cả các phương thức được đặt tên theo quy tắc snake_case:

- Phần đầu tên là động từ mô tả hoạt động chính của hàm (Ví dụ: `load_`, `display_`, `handle_`, ...).
- Phần sau tên mô tả đối tượng mà hoạt động của hàm tác động (Ví dụ: `_main_menu`, `_choice_1`, ...).

```
def display_main_menu(self):
```

Đối với các file module cũng vậy, tên được đặt gợi tả chức năng mà module ấy thực hiện nhưng em đặt tên chúng đều là danh từ để mô tả các

module như các khối chức năng bộ phận của một tổng thể, tên được đặt theo quy tắc snake_case.

Đối với tên các lớp đối tượng (class), tên được đặt đều là danh từ nhưng lại tuân thủ theo một quy tắc khác là PascalCase để tránh việc nhầm lẫn chúng với tên các biến.

```
class AttendanceRecord:
    """Class này biểu diễn đối tượng Bản ghi điểm danh"""
```

Chú thích trong chương trình:

- Docstring ở đầu các module, class và hàm.

```
def display_attendance_records(records: List[AttendanceRecord],
                               title: Optional[str] = None):
    """
    Đầu vào:
    - records (kiểu List[AttendanceRecord]): Danh sách các bản ghi cần hiện.
    - title (kiểu Optional[str]): Tiêu đề tùy chọn cho bảng kết quả.
    Đầu ra: Không có.
    Hàm này hiển thị một danh sách các bản ghi điểm danh và
    có thể tùy chỉnh tiêu đề.
    """
    if not records:
        print("Không có bản ghi điểm danh nào phù hợp để hiển thị.")
        return
```

- Comment cho các đoạn chương trình.

```
# Nhập trạng thái điểm danh
status_input = input(f" - {student.full_name} ({student.student_id}): ").strip()
resolved_status = ""

# Nếu người dùng không nhập thì coi như "Có mặt"
if not status_input:
    resolved_status = "Có mặt"
else:
    # Chuẩn hóa và xác nhận trạng thái nhập vào
    normalized_char = status_input.upper()

    # Nếu người dùng nhập chữ cái tắt thì ánh xạ sang trạng thái đầy đủ
    if normalized_char == 'C': resolved_status = "Có mặt"
    elif normalized_char == 'V': resolved_status = "Vắng"
    elif normalized_char == 'CP': resolved_status = "Có phép"
```

3.2.2 Trình bày mã trong chương trình

Khi trình bày mã trong chương trình, em đã tuân theo một vài yêu cầu trong tiêu chuẩn hướng dẫn viết code “sạch” PEP 8 Python để căn lề, cách đoạn:

- Em đã sử dụng bốn dấu cách (khoảng trắng) cho mỗi cấp thụt lề sau khi xuống dòng.

```
def get_class_by_id(class_id: str) -> Optional[Class]:
    """
    Đầu vào: class_id (kiểu str): Mã lớp học cần tìm.
    Đầu ra: Một đối tượng Class nếu tìm thấy, ngược lại trả về None
    Hàm này tìm kiếm và trả về một đối tượng Class
    | từ cache dựa trên mã lớp học được cung cấp.
    """
    load_all_data_once()
    for school_class in ALL_CLASSES:
        if school_class.class_id == class_id: # So sánh chính xác
            return school_class
    return None
```

- Cách hai dòng cho mỗi lớp đối tượng, cách một dòng cho các hàm nằm trong class hoặc giữa các khối lệnh điều kiện, vòng lặp hoặc khối lệnh thực hiện chức năng riêng biệt.

```
✓ def __repr__(self) -> str:
✓     """
        Đầu vào: self - tham chiếu đến đối tượng (ngầm định).
        Đầu ra: Không có
        Phương thức này trả về chuỗi kỹ thuật thể hiện cách tạo lại đối
        | dùng cho debug
        """
        return (f"Class(class_id='{self.class_id}', "
                f"class_name='{self.class_name}')"

✓
✓ def change_object_to_dict(self) -> dict:
        """
        Đầu vào: self - tham chiếu đến đối tượng (ngầm định).
        Đầu ra: Một dictionary tương ứng với đối tượng
        Phương thức này chuyển đổi đối tượng Class thành 1 dictionary
        """
        return {
            "class_id": self.class_id,
            "class_name": self.class_name,
            "student_ids": self.student_ids
        }

✓ class AttendanceRecord:
```

CHƯƠNG 4. GỠ LỖI VÀ KIỂM THỬ

4.1 Xác định & Gỡ lỗi

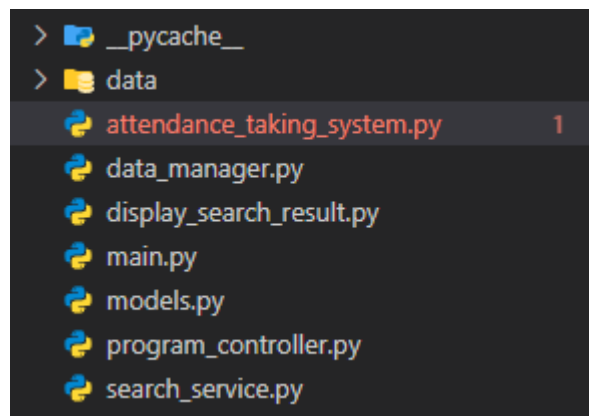
Tìm kiếm và xác định lỗi là quá trình xác định xem chương trình có lỗi (bug) không, bằng cách:

- Quan sát kết quả sai.
- So sánh với đầu ra mong đợi.
- Kiểm thử các tình huống bất thường.

Việc tìm kiếm lỗi cú pháp trong quá trình viết chương trình là tương đối dễ dàng vì Python sẽ hiển thị lên màn hình ngay. Ví dụ như đoạn mã này ở dòng lệnh thứ hai gặp lỗi thụt lề, python sẽ báo ngay lập tức trong đoạn code.

```
# Lặp qua từng SV trong lớp để nhập trạng thái điểm danh
student_ids_in_class = school_class.get_student_ids() # Lấy danh sách SV trong lớp
if not student_ids_in_class:
```

Không chỉ vậy, IDE cũng thông báo lỗi cú pháp ở file nào trong thư mục được mở và số lượng lỗi trong file đó ngay trên màn hình:



Đối với lỗi ngữ nghĩa thì khó phát hiện hơn lỗi cú pháp do cần phải được debug hoặc kiểm thử mới có thể phát hiện ra.

Debug (Gỡ lỗi) là quá trình tìm nguyên nhân gây ra lỗi và sửa nó để chương trình hoạt động đúng. Dưới đây là quy trình gỡ lỗi cơ bản:

- Xác định nơi xảy ra lỗi: Khi ấn vào nút Run trên thanh tác vụ của IDE, kết quả thực thi chương trình và cả thông báo lỗi sẽ hiển thị bên trong Terminal. Python thông báo lỗi và traceback rất chi tiết: có hiển thị loại lỗi và cả vị trí dòng lệnh xảy ra lỗi.
- Xác định loại lỗi: Debugger có thể xác định các loại lỗi sau: TypeError, ValueError, AttributeError, KeyError...
- Hiển thị thông báo lỗi một cách dễ hiểu bên phải loại lỗi.

Thực tế, trong quá trình viết mã nguồn chương trình, em đã gặp phải nhiều tình huống lỗi cú pháp lẫn lỗi logic. Ngoài việc sử dụng công cụ debugger của IDE để theo dõi luồng chạy, điểm dừng... thì em có thể hạn chế và phát hiện lỗi qua các cách thức khác như:

- Phân tách chương trình thành các phần nhỏ để dễ dàng kiểm tra từng phần.
- Viết mã rõ ràng, dễ theo dõi để giúp dễ phát hiện lỗi hơn.
- Kiểm tra mã mới nhất: Luôn đảm bảo đang làm việc với phiên bản mã mới nhất, tránh debug nhằm phiên bản cũ. Điều này tương đối dễ thực hiện chỉ bằng việc nhấn thao tác File\Save sau mỗi lần cập nhật file mã nguồn.
- Sử dụng các cấu trúc try – except để định vị các loại lỗi tiềm ẩn.
- Phát hiện và sửa lỗi (sửa mã nguồn) ngay trong khi kiểm thử.

Trải qua nhiều phiên bản khác nhau và sửa các lỗi ở các hàm, vòng lặp; cuối cùng mới có thể đưa ra mã nguồn chương trình cuối cùng tối ưu và tránh các lỗi sai nhất.

4.2 Kiểm thử

Kiểm thử là quá trình đánh giá một chương trình hoặc hệ thống để xác định xem nó có hoạt động đúng theo yêu cầu, phát hiện lỗi và đảm bảo chất lượng của chương trình.

Mục tiêu chính:

- Phát hiện bug trước khi đến tay người dùng.
- Đảm bảo chức năng chạy đúng như thiết kế.
- Kiểm tra xem phần mềm có đạt được hiệu suất, bảo mật, khả năng mở rộng như mong muốn không...

Các phương pháp kiểm thử có thể kể ra như:

- Kiểm thử hộp đen (Black-box Testing): Tập trung kiểm tra đầu vào và đầu ra của chương trình.
- Kiểm thử hộp trắng (White-box Testing): Tập trung kiểm tra các cấu trúc logic của mã nguồn bằng cách chạy các case kiểm thử này và so sánh kết quả với kết quả mong đợi, có thể xác định chương trình có chạy đúng không.

38 test case mà em thực hiện trong chương trình chứa đựng những khía cạnh của cả hai phương pháp kiểm thử hộp đen và hộp trắng:

- Một vài khía cạnh của kiểm thử hộp đen:
 - + Tồn tại những test case xác minh, đặc tả sáu chức năng chính hoạt động đúng theo yêu cầu, mong đợi của người dùng.
 - + Các test case tập trung vào input được cung cấp và output nhận được và so sánh nó với output mong đợi, không cần đào sâu vào chi tiết triển khai.
 - + Các test case mô phỏng kịch bản sử dụng của người dùng để kiểm tra sự phối hợp của các module từ đầu đến cuối.
- Một vài khía cạnh của kiểm thử hộp trắng:

- + Chủ động tạo ra các input để kích hoạt các nhánh if/else và try-except cụ thể. Vì vậy mà tồn tại những test case kiểm tra các trường hợp đầu vào rỗng, đầu vào không hợp lệ, sai định dạng cho đầu vào.
- + Kiểm tra các thao tác logic trong các hàm tích hợp trong chức năng và trả về thông báo lỗi nếu một thao tác nào đó thực thi không thành công (Ví dụ như trả về thông báo khi chưa có bản ghi điểm danh nào trong file trước khi thực hiện điểm danh lần đầu tiên...).

Bên cạnh đó, em cũng sử dụng chiến lược kiểm thử hồi quy (Regression Testing). Cụ thể hơn, sau mỗi lần sửa lỗi và cải tiến chương trình – ví dụ như thêm chức năng mới, em phải kiểm thử lại bằng việc chạy các test case của chức năng cũ để đảm bảo việc thêm mới không gây ra xung đột giữa các luồng thực hiện chức năng, có thể làm hỏng chương trình.

CHƯƠNG 5. CÁC TÌNH HUỐNG KIỂM THỬ

5.1 Kiểm thử menu chính

5.1.1 Test case 1: Hiển thị menu chính

- Nội dung: In ra màn hình danh sách các chức năng của chương trình.
- Input: Không có
- Output mong đợi: Menu chính với các lựa chọn từ 1 đến 6 và 0 được hiển thị đúng. Thông báo tải dữ liệu ban đầu xuất hiện.
- Thực hiện và kết quả:

```
Đang khởi tạo hệ thống và tải dữ liệu...  
Dữ liệu cơ bản (Sinh viên, Lớp học) đã được xử lý.  
Thông báo: Chưa có bản ghi điểm danh nào.  
  
--- HỆ THỐNG ĐIỂM DANH (MVC) ---  
1. Điểm danh cho lớp học  
2. Cập nhật trạng thái điểm danh  
3. Tìm kiếm sinh viên  
4. Tìm kiếm lớp học  
5. Xem lịch sử điểm danh của sinh viên  
6. Xem báo cáo điểm danh của lớp  
0. Thoát  
Nhập lựa chọn của bạn:
```

5.1.2 Test case 2: Lựa chọn không hợp lệ (vượt khoảng giá trị menu)

- Nội dung: Từ menu chính, nhập một số không có trong danh sách.
- Input: 7.
- Output mong đợi: Thông báo “Lựa chọn không hợp lệ. Vui lòng chọn lại.”
- Thực hiện và kết quả:

```
--- HỆ THỐNG ĐIỂM DANH (MVC) ---  
1. Điểm danh cho lớp học  
2. Cập nhật trạng thái điểm danh  
3. Tìm kiếm sinh viên  
4. Tìm kiếm lớp học  
5. Xem lịch sử điểm danh của sinh viên  
6. Xem báo cáo điểm danh của lớp  
0. Thoát  
Nhập lựa chọn của bạn: 7  
Lựa chọn không hợp lệ. Vui lòng chọn lại.  
  
Nhấn Enter để tiếp tục...[]
```

5.1.3 Test case 3: Lựa chọn đầu vào không phải số

- Nội dung: Từ menu chính, Từ menu chính, nhập một chữ cái hoặc ký tự đặc biệt.

- Input: XYZ.
- Output mong đợi: Thông báo “Lựa chọn không hợp lệ. Vui lòng chọn lại.”
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: XYZ
Lựa chọn không hợp lệ. Vui lòng chọn lại.

Nhấn Enter để tiếp tục...

```

5.1.4 Test case 4: Lựa chọn thoát chương trình

- Nội dung: Từ menu chính, chọn “Thoát”
- Input: 0.
- Output mong đợi: “Đang thoát chương trình...”
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 0
Đang thoát chương trình...

```

5.2 Kiểm thử chức năng 1: Điểm danh cho lớp học

5.2.1 Test case 5: Điểm danh thành công cho lớp học có sinh viên

- Nội dung: Chọn chức năng 1, điểm danh cho một lớp học hợp lệ, ngày hợp lệ, và nhập trạng thái hợp lệ cho sinh viên.
- Input: 1 (Lựa chọn menu), DTB01 (Mã lớp học), để trống (Ngày điểm danh là hôm nay).
- Output mong đợi:
 - + “Điểm danh thành công và đã lưu kết quả.”.
 - + Các bản ghi điểm danh vừa được thực hiện xử lý khi chạy chương trình được ghi vào file attendance_records.json.
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 1

Danh sách các lớp học hiện có:

--- DANH SÁCH LỚP HỌC ---
- Mã lớp: DTB01, Tên lớp: Database 01, Sĩ số: 22
- Mã lớp: DATA01, Tên lớp: CTDL&GT 01, Sĩ số: 30
- Mã lớp: CODETECH01, Tên lớp: KTLT 01, Sĩ số: 30
- Mã lớp: DTB02, Tên lớp: Database 02, Sĩ số: 0
Nhập mã lớp học cần điểm danh: DTB01
Nhập ngày điểm danh (YYYY-MM-DD, để trống là hôm nay - 2025-06-08):

--- Điểm danh lớp: DTB01 ---
Ngày: 2025-06-08
Trạng thái hợp lệ: Có mặt, Vắng, Có phép
Nhập trạng thái cho từng sinh viên:
- Nguyễn Văn A (SV101): C
- Trần Văn B (SV102): C
- Hoàng Chí C (SV104): C
- Đoàn Ngọc D (SV105): C
- Bùi Đức H (SV106): C
- Lê Minh H (SV108): V
- Đào Trung H (SV109): V
- Hoàng Thái L (SV112): C
- Hoàng Bình N (SV115): C
- Đỗ Mạnh N (SV116): V
- Đào Tiến N (SV117): CP
- Nguyễn Ngọc O (SV118): CP
- Dương Thảo Q (SV119): V
- Phạm Thị Q (SV120): V
- Nông Như T (SV121): C
- Phạm Khánh T (SV122): C
- Nguyễn Ngọc V (SV125): C
- Trần Khánh V (SV126): C
- Phạm Xuân V (SV127): C
- Kim Duy V (SV128): C
- Trần Long V (SV129): C
- Vũ Anh X (SV130): C
Điểm danh thành công và đã lưu kết quả.

```

Kết quả hiển thị trên giao diện dòng lệnh

```

[
  {
    "student_id": "SV101",
    "class_id": "DTB01",
    "attendance_date": "2025-06-08",
    "status": "Có mặt"
  },

```

Kết quả hiển thị trong file attendance_records.json (Lấy mẫu một bản ghi)

5.2.2 Test case 6: Điểm danh cho lớp không tồn tại

- Nội dung: Chọn chức năng 1, nhập mã lớp không có trong hệ thống.
- Input: 1 (Lựa chọn menu), DTB03 (Mã lớp học)
- Output mong đợi: “Lớp học với mã ‘DTB03’ không tồn tại.”
- Thực hiện và kết quả:

```
--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 1

Danh sách các lớp học hiện có:

--- DANH SÁCH LỚP HỌC ---
- Mã lớp: DTB01, Tên lớp: Database 01, Sĩ số: 22
- Mã lớp: DATA01, Tên lớp: CTDL&GT 01, Sĩ số: 30
- Mã lớp: CODETECH01, Tên lớp: KTLT 01, Sĩ số: 30
- Mã lớp: DTB02, Tên lớp: Database 02, Sĩ số: 0
Nhập mã lớp học cần điểm danh: DTB03
Lớp học với mã 'DTB03' không tồn tại.
```

5.2.3 Test case 7: Điểm danh với mã lớp để trống

- Nội dung: Chọn chức năng 1, để trống mã lớp.
- Input: 1 (Lựa chọn menu), để trống (Mã lớp học)
- Output mong đợi: “Mã lớp không được để trống.”
- Thực hiện và kết quả:

```
--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 1

Danh sách các lớp học hiện có:

--- DANH SÁCH LỚP HỌC ---
- Mã lớp: DTB01, Tên lớp: Database 01, Sĩ số: 22
- Mã lớp: DATA01, Tên lớp: CTDL&GT 01, Sĩ số: 30
- Mã lớp: CODETECH01, Tên lớp: KTLT 01, Sĩ số: 30
- Mã lớp: DTB02, Tên lớp: Database 02, Sĩ số: 0
Nhập mã lớp học cần điểm danh:
Mã lớp không được để trống.
```

5.2.4 Test case 8: Điểm danh với đầu vào ngày sai định dạng

- Nội dung: Chọn chức năng 1, chọn mã lớp và nhập ngày điểm danh.
- Input: 1 (Lựa chọn menu), DTB01 (Mã lớp học), 10-06-2025 (Ngày điểm danh)
- Output mong đợi: “Định dạng ngày không hợp lệ. Vui lòng nhập lại theo YYYY-MM-DD”.
- Thực hiện và kết quả:

```
--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 1

Danh sách các lớp học hiện có:

--- DANH SÁCH LỚP HỌC ---
- Mã lớp: DTB01, Tên lớp: Database 01, Sĩ số: 22
- Mã lớp: DATA01, Tên lớp: CTDL&GT 01, Sĩ số: 30
- Mã lớp: CODETECH01, Tên lớp: KTLT 01, Sĩ số: 30
- Mã lớp: DTB02, Tên lớp: Database 02, Sĩ số: 0
Nhập mã lớp học cần điểm danh: DTB01
Nhập ngày điểm danh (YYYY-MM-DD, để trống là hôm nay - 2025-06-08): 10-06-2025
Định dạng ngày không hợp lệ. Vui lòng nhập lại theo YYYY-MM-DD.
Nhập ngày điểm danh (YYYY-MM-DD, để trống là hôm nay - 2025-06-08): 
```

5.2.5 Test case 9: Điểm danh với lớp không có sinh viên

- Nội dung: Chọn chức năng 1, chọn mã lớp mà có sĩ số bằng 0.
- Input: 1 (Lựa chọn menu), DTB02 (Mã lớp học), để trống (Ngày điểm danh là hôm nay)
- Output mong đợi: “Thông báo: Lớp học này chưa có sinh viên nào.”.
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 1

Danh sách các lớp học hiện có:

--- DANH SÁCH LỚP HỌC ---
- Mã lớp: DTB01, Tên lớp: Database 01, Sĩ số: 22
- Mã lớp: DATA01, Tên lớp: CTDL&GT 01, Sĩ số: 30
- Mã lớp: CODETECH01, Tên lớp: KTLT 01, Sĩ số: 30
- Mã lớp: DTB02, Tên lớp: Database 02, Sĩ số: 0
Nhập mã lớp học cần điểm danh: DTB02
Nhập ngày điểm danh (YYYY-MM-DD, để trống là hôm nay - 2025-06-08):

--- Điểm danh lớp: DTB02 ---
Ngày: 2025-06-08
Trạng thái hợp lệ: Có mặt, Vắng, Có phép
Nhập trạng thái cho từng sinh viên:
Thông báo: Lớp học này chưa có sinh viên nào.

```

5.2.6 Test case 10: Nhập trạng thái không hợp lệ cho sinh viên

- Nội dung: Trong quá trình điểm danh, nhập một trạng thái không hợp lệ
- Input: 1 (Lựa chọn menu), DTB01 (Mã lớp học), 2025-06-09 (Ngày điểm danh)
- Output mong đợi: “Trạng thái không hợp lệ...”.
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 1

Danh sách các lớp học hiện có:

--- DANH SÁCH LỚP HỌC ---
- Mã lớp: DTB01, Tên lớp: Database 01, Sĩ số: 22
- Mã lớp: DATA01, Tên lớp: CTDL&GT 01, Sĩ số: 30
- Mã lớp: CODETECH01, Tên lớp: KTLT 01, Sĩ số: 30
- Mã lớp: DTB02, Tên lớp: Database 02, Sĩ số: 0
Nhập mã lớp học cần điểm danh: DTB01
Nhập ngày điểm danh (YYYY-MM-DD, để trống là hôm nay - 2025-06-08): 2025-06-09

--- Điểm danh lớp: DTB01 ---
Ngày: 2025-06-09
Trạng thái hợp lệ: Có mặt, Vắng, Có phép
Nhập trạng thái cho từng sinh viên:
- Nguyễn Văn A (SV101): abcdefu
Trạng thái 'abcdefu' không hợp lệ.
Vui lòng nhập một trong: Có mặt, Vắng, Có phép hoặc C, V, CP.
- Nguyễn Văn A (SV101): 

```

5.2.7 Test case 11: Điểm danh lại cho một buổi điểm danh (ghi đề)

- Nội dung: Thực hiện lại test case 5 với cùng lớp và cùng ngày đó, nhưng với trạng thái khác cho một vài sinh viên.
- Input: 1 (Lựa chọn menu), DTB01 (Mã lớp học), để trống (Ngày điểm danh là hôm nay)
- Output mong đợi:
 - + “Điểm danh thành công và đã lưu kết quả.”.
 - + Các bản ghi điểm danh trong file attendance_records.json thay đổi.
- Thực hiện và kết quả:

```
--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 1

Danh sách các lớp học hiện có:

--- DANH SÁCH LỚP HỌC ---
- Mã lớp: DTB01, Tên lớp: Database 01, Sĩ số: 22
- Mã lớp: DATA01, Tên lớp: CTDL&GT 01, Sĩ số: 30
- Mã lớp: CODETECH01, Tên lớp: KTLT 01, Sĩ số: 30
- Mã lớp: DTB02, Tên lớp: Database 02, Sĩ số: 0
Nhập mã lớp học cần điểm danh: DTB01
Nhập ngày điểm danh (YYYY-MM-DD, để trống là hôm nay - 2025-06-08):

--- Điểm danh lớp: DTB01 ---
Ngày: 2025-06-08
Trạng thái hợp lệ: Có mặt, Vắng, Có phép
Nhập trạng thái cho từng sinh viên:
- Nguyễn Văn A (SV101): V
- Trần Văn B (SV102): V
- Hoàng Chí C (SV104): CP
- Đoàn Ngọc D (SV105): CP
- Bùi Đức H (SV106): V
- Lê Minh H (SV108): C
- Đào Trung H (SV109): C
- Hoàng Thái L (SV112): C
- Hoàng Bình N (SV115): C
- Đỗ Mạnh N (SV116): C
- Đào Tiến N (SV117): C
- Nguyễn Ngọc O (SV118): C
- Dương Thảo Q (SV119): C
- Phạm Thị Q (SV120): C
- Nông Như T (SV121): C
- Phạm Khánh T (SV122): C
- Nguyễn Ngọc V (SV125): C
- Trần Khánh V (SV126): C
- Phạm Xuân V (SV127): C
- Kim Duy V (SV128): C
- Trần Long V (SV129): C
- Vũ Anh X (SV130): C
Điểm danh thành công và đã lưu kết quả.
```

Kết quả hiển thị trên giao diện dòng lệnh

```
[
  {
    "student_id": "SV101",
    "class_id": "DTB01",
    "attendance_date": "2025-06-08",
    "status": "Vắng"
  },
]
```

Kết quả hiển thị trong file attendance_records.json đã được thay đổi (Lấy mẫu một bản ghi)

5.3 Kiểm thử chức năng 2: Cập nhật trạng thái điểm danh

5.3.1 Test case 12: Cập nhật điểm danh thành công

- Nội dung: Chọn chức năng 2, cập nhật trạng thái cho một bản ghi điểm danh đã tồn tại.
- Input: 2 (Lựa chọn menu), SV102 (Mã sinh viên), DTB01 (Mã lớp học), 2025-06-08 (Ngày điểm danh),
- Output mong đợi:
 - + “Đã cập nhật trạng thái điểm danh cho SV SV102 tại lớp DTB01 ngày 2025-06-08...”.
 - + Bản ghi điểm danh của sinh viên đó trong file attendance_records.json thay đổi
- Thực hiện và kết quả:

```
--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 2
Nhập mã sinh viên: SV102
Nhập mã lớp học: DTB01
Nhập ngày điểm danh đã ghi (YYYY-MM-DD): 2025-06-08
Nhập trạng thái mới (Có mặt, Vắng, Có phép) hoặc chữ cái viết tắt. CP
Đã cập nhật trạng thái điểm danh cho SV SV102 tại lớp DTB01 ngày 2025-06-08 từ 'Vắng' thành 'Có phép'.
```

Kết quả hiển thị trên giao diện dòng lệnh

```
{
  "student_id": "SV102",
  "class_id": "DTB01",
  "attendance_date": "2025-06-08",
  "status": "Có phép"
},
```

Kết quả hiển thị trong file attendance_records.json đã được thay đổi (Lấy mẫu một bản ghi)

5.3.2 Test case 13: Cập nhật bản ghi điểm danh không tồn tại

- Nội dung: Cố gắng cập nhật một bản ghi không có trong hệ thống.
- Input: 2 (Lựa chọn menu), SV101 (Mã sinh viên), DATA01 (Mã lớp học), 2025-06-08 (Ngày điểm danh),

- Output mong đợi: “Lỗi: Không tìm thấy bản ghi ... để cập nhật.”
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 2
Nhập mã sinh viên: SV101
Nhập mã lớp học: DATA01
Nhập ngày điểm danh đã ghi (YYYY-MM-DD): 2025-06-08
Nhập trạng thái mới (Có mặt, Vắng, Có phép) hoặc chữ cái viết tắt. C
Lỗi: Không tìm thấy bản ghi điểm danh cho SV SV101 tại lớp DATA01 ngày 2025-06-08 để cập nhật.

```

5.3.3 Test case 14: Cập nhật với trạng thái mới không hợp lệ

- Nội dung: Cố gắng cập nhật một bản ghi với trạng thái mới không hợp lệ.
- Input: 2 (Lựa chọn menu), SV129 (Mã sinh viên), DTB01 (Mã lớp học), 2025-06-08 (Ngày điểm danh), “Không đi học” (Trạng thái mới).
- Output mong đợi: “Trạng thái mới “Không đi học” không hợp lệ...”
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 2
Nhập mã sinh viên: SV129
Nhập mã lớp học: DTB01
Nhập ngày điểm danh đã ghi (YYYY-MM-DD): 2025-06-08
Nhập trạng thái mới (Có mặt, Vắng, Có phép) hoặc chữ cái viết tắt: Không đi học
Lỗi: Trạng thái mới 'Không đi học' không hợp lệ. Trạng thái cần nhập: Có mặt, Vắng, Có phép hoặc C, V, CP.
Nhấn Enter để tiếp tục...

```

5.3.4 Test case 15: Cập nhật trạng thái giống trạng thái cũ

- Nội dung: Cập nhật trạng thái thành giá trị hiện tại của nó.
- Input: 2 (Lựa chọn menu), SV130 (Mã sinh viên), DTB01 (Mã lớp học), 2025-06-08 (Ngày điểm danh), “Có mặt” (Trạng thái mới – giống trạng thái cũ).
- Output mong đợi: “Trạng thái mới “Không đi học” không hợp lệ...”
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 2
Nhập mã sinh viên: SV130
Nhập mã lớp học: DTB01
Nhập ngày điểm danh đã ghi (YYYY-MM-DD): 2025-06-08
Nhập trạng thái mới (Có mặt, Vắng, Có phép) hoặc chữ cái viết tắt: C
Trạng thái của SV SV130 đã là 'Có mặt' không cần cập nhật.

Nhấn Enter để tiếp tục...

```

Kết quả hiển thị trên giao diện dòng lệnh

```

{
  "student_id": "SV130",
  "class_id": "DTB01",
  "attendance_date": "2025-06-08",
  "status": "Có mặt"
}

```

*Kết quả hiển thị trong file attendance_records.json
(Lấy mẫu một bản ghi – không thay đổi trạng thái)*

5.3.5 Test case 16: Để trống các trường input bắt buộc

- Nội dung: Thử để trống các trường đầu vào được yêu cầu.
- Input: 2 (Lựa chọn menu), để trống một trong các trường input được yêu cầu.
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 2
Nhập mã sinh viên:
Nhập mã lớp học:
Sinh viên với mã '' không tồn tại.

```

Để trống trường mã sinh viên.

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 2
Nhập mã sinh viên: SV125
Nhập mã lớp học:
Lớp học với mã '' không tồn tại.

```

Để trống trường mã lớp học.

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 2
Nhập mã sinh viên: SV128
Nhập mã lớp học: DTB01
Nhập ngày điểm danh đã ghi (YYYY-MM-DD):
Ngày điểm danh không được để trống.

```

Để trống trường ngày điểm danh.

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 2
Nhập mã sinh viên: SV128
Nhập mã lớp học: DTB01
Nhập ngày điểm danh đã ghi (YYYY-MM-DD): 2025-06-08
Nhập trạng thái mới (Có mặt, Vắng, Có phép) hoặc chữ cái viết tắt:
Tất cả các trường thông tin không được để trống.

```

Kết quả tổng quát: Vì để trống các trường kia đều có thông báo riêng nên thông báo này chỉ hiển thị nếu để trống trường trạng thái mới.

5.4 Kiểm thử chức năng 3: Tìm kiếm sinh viên

5.4.1 Test case 17: Tìm kiếm chính xác theo mã sinh viên

- Nội dung: Tìm kiếm mã sinh viên có phân biệt chữ hoa/thường (nhập chữ hoa).
- Input: 3 (Lựa chọn menu), SV127 (Mã sinh viên).
- Output mong đợi: “--- SINH VIÊN TÌM THẤY --- ...”
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 3
Nhập tên hoặc mã sinh viên cần tìm: SV127

--- SINH VIÊN TÌM THẤY ---
- Mã SV: SV127, Tên: Phạm Xuân V

```

5.4.2 Test case 18: Tìm kiếm mã sinh viên sai quy tắc nhập

- Nội dung: Tìm kiếm mã sinh viên có phân biệt chữ hoa/thường (nhập chữ thường).

- Input: 3 (Lựa chọn menu), sv127 (Mã sinh viên).
- Output mong đợi: “Không tìm thấy sinh viên nào phù hợp.”.
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 3
Nhập tên hoặc mã sinh viên cần tìm: sv127
Không tìm thấy sinh viên nào phù hợp.

```

5.4.3 Test case 19: Tìm kiếm sinh viên theo tên đầy đủ

- Nội dung: Tìm kiếm sinh viên theo họ tên (không phân biệt chữ hoa/thường).
- Input: 3 (Lựa chọn menu), Nguyễn văn A (Tên sinh viên).
- Output mong đợi: “--- SINH VIÊN TÌM THẤY --- ...”.
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 3
Nhập tên hoặc mã sinh viên cần tìm: Nguyễn văn A

--- SINH VIÊN TÌM THẤY ---
- Mã SV: SV101, Tên: Nguyễn Văn A

```

5.4.4 Test case 20: Tìm kiếm sinh viên theo một phần tên

- Nội dung: Tìm kiếm sinh viên theo một phần tên gọi (không phân biệt chữ hoa/thường).
- Input: 3 (Lựa chọn menu), Hoàng chí (Một phần tên sinh viên).
- Output mong đợi: “--- SINH VIÊN TÌM THẤY --- ...”.
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 3
Nhập tên hoặc mã sinh viên cần tìm: Hoàng chí

--- SINH VIÊN TÌM THẤY ---
- Mã SV: SV104, Tên: Hoàng Chí C

```

5.4.5 Test case 21: Tìm kiếm sinh viên với tên hoặc mã sai định dạng

- Nội dung: Tìm kiếm sinh viên theo nhưng input sai định dạng
- Input: 3 (Lựa chọn menu), 12345 (Mã sinh viên).
- Output mong đợi: “Không tìm thấy sinh viên nào phù hợp”.
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 3
Nhập tên hoặc mã sinh viên cần tìm: 12345
Không tìm thấy sinh viên nào phù hợp.

```

5.4.6 Test case 22: Tìm kiếm sinh viên với input rỗng

- Nội dung: Tìm kiếm sinh viên theo nhưng input rỗng.
- Input: 3 (Lựa chọn menu), để trống (Tên hoặc mã sinh viên).
- Output mong đợi: “Nội dung tìm kiếm không được để trống.”.
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 3
Nhập tên hoặc mã sinh viên cần tìm:
Nội dung tìm kiếm không được để trống.

```

5.5 Kiểm thử chức năng 4: Tìm kiếm lớp học

5.5.1 Test case 23: Tìm kiếm chính xác theo mã lớp học

- Nội dung: Tìm kiếm lớp học theo mã lớp (có phân biệt chữ hoa/thường – nhập chữ hoa).
- Input: 4 (Lựa chọn menu), DTB02 (Mã lớp học).
- Output mong đợi: “--- DANH SÁCH LỚP HỌC ---....”.
- Thực hiện và kết quả:

```
--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 4
Nhập tên hoặc mã lớp cần tìm: DTB02

--- DANH SÁCH LỚP HỌC ---
- Mã lớp: DTB02, Tên lớp: Database 02, Sĩ số: 0
```

5.5.2 Test case 24: Tìm kiếm sai mã lớp học

- Nội dung: Tìm kiếm lớp học theo mã lớp (có phân biệt chữ hoa/thường – nhập chữ thường).
- Input: 4 (Lựa chọn menu), dtb02 (Mã lớp học).
- Output mong đợi: “Không tìm thấy lớp học nào.”
- Thực hiện và kết quả:

```
--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 4
Nhập tên hoặc mã lớp cần tìm: dtb02
Không tìm thấy lớp học nào.
```

5.5.3 Test case 25: Tìm kiếm lớp học theo tên lớp

- Nội dung: Tìm kiếm lớp học theo tên lớp (không phân biệt chữ hoa/thường).
- Input: 4 (Lựa chọn menu), database 01 (Tên lớp học).
- Output mong đợi: “--- DANH SÁCH LỚP HỌC ---....”.
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 4
Nhập tên hoặc mã lớp cần tìm: database 01

--- DANH SÁCH LỚP HỌC ---
- Mã lớp: DTB01, Tên lớp: Database 01, Sĩ số: 22

```

5.5.4 Test case 26: Tìm kiếm theo một phần tên lớp học

- Nội dung: Tìm kiếm lớp học theo một phần tên lớp (không phân biệt chữ hoa/thường).
- Input: 4 (Lựa chọn menu), Database (Tên lớp học).
- Output mong đợi: “--- DANH SÁCH LỚP HỌC ---....”.
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 4
Nhập tên hoặc mã lớp cần tìm: Database

--- DANH SÁCH LỚP HỌC ---
- Mã lớp: DTB01, Tên lớp: Database 01, Sĩ số: 22
- Mã lớp: DTB02, Tên lớp: Database 02, Sĩ số: 0

```

5.5.5 Test case 27: Tìm kiếm lớp học không có trong danh sách

- Nội dung: Tìm kiếm lớp học không có trong file đầu vào.
- Input: 4 (Lựa chọn menu), Đại số 01 (Tên lớp học).
- Output mong đợi: “Không tìm thấy lớp học nào.”.
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 4
Nhập tên hoặc mã lớp cần tìm: Đại số 01
Không tìm thấy lớp học nào.

```


5.5.6 Test case 28: Tìm kiếm lớp học với input rỗng

- Nội dung: Tìm kiếm lớp học mà không nhập input.
- Input: 4 (Lựa chọn menu), để trống (Mã hoặc Tên lớp học).
- Output mong đợi: “Nội dung tìm kiếm không được để trống.”.
- Thực hiện và kết quả:

```
--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 4
Nhập tên hoặc mã lớp cần tìm:
Nội dung tìm kiếm không được để trống.
```

5.6 Kiểm thử chức năng 5: Xem lịch sử điểm danh của sinh viên

5.6.1 Test case 29: Xem lịch sử của SV, không lọc lớp

- Nội dung:
 - + Trước khi thực hiện test case này, em đã thực hiện lại trường hợp test case 1 – điểm danh với một lớp học khác có mã DATA01, cùng ngày 2025-06-08.
 - + Xem lịch sử điểm danh của sinh viên bất kỳ trong toàn bộ lớp học có sinh viên đó.
- Input: 5 (Lựa chọn menu), SV101 (Mã sinh viên) để trống (Mã lớp học).
- Output mong đợi: “--- LỊCH SỬ ĐIỂM DANH CỦA SINH VIÊN”.
- Thực hiện và kết quả:

```
--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 5
Nhập mã sinh viên để xem lịch sử điểm danh: SV101
Nhập mã lớp để lọc (để trống để xem tất cả các lớp):

--- LỊCH SỬ ĐIỂM DANH CỦA SINH VIÊN NGUYỄN VĂN A (SV101) ---
Ngày      | Mã Lớp | Tên Lớp      | Mã SV | Tên SV      | Trạng thái
-----|-----|-----|-----|-----|-----
2025-06-08 | DTB01  | Database 01  | SV101 | Nguyễn Văn A | Vắng
2025-06-08 | DATA01 | CTDL&GT 01  | SV101 | Nguyễn Văn A | Có mặt
```

5.6.2 Test case 30: Xem lịch sử của SV, có lọc lớp hợp lệ

- Nội dung: Xem lịch sử điểm danh của sinh viên bất kỳ trong một lớp học bất kỳ.
- Input: 5 (Lựa chọn menu), SV101 (Mã sinh viên), DATA01 (Mã lớp học).

- Output mong đợi: “--- LỊCH SỬ ĐIỂM DANH CỦA SINH VIÊN – LỚP ...”.
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 5
Nhập mã sinh viên để xem lịch sử điểm danh: SV101
Nhập mã lớp để lọc (để trống để xem tất cả các lớp): DATA01

--- LỊCH SỬ ĐIỂM DANH CỦA SINH VIÊN NGUYỄN VĂN A (SV101) - LỚP: CTDL&GT 01 (DATA01) ---

```

Ngày	Mã Lớp	Tên Lớp	Mã SV	Tên SV	Trạng thái
2025-06-08	DATA01	CTDL> 01	SV101	Nguyễn Văn A	Có mặt

5.6.3 Test case 31: Xem lịch sử của SV; có lọc lớp, chưa điểm danh

- Nội dung: Xem lịch sử điểm danh của sinh viên bất kỳ trong một lớp học bất kỳ (chưa điểm danh).
- Input: 5 (Lựa chọn menu), SV101 (Mã sinh viên), CODETECH01 (Mã lớp học).
- Output mong đợi: “Không có bản ghi nào phù hợp để hiển thị.”.
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 5
Nhập mã sinh viên để xem lịch sử điểm danh: SV101
Nhập mã lớp để lọc (để trống để xem tất cả các lớp): CODETECH01
Không có bản ghi điểm danh nào phù hợp để hiển thị.

```

5.6.4 Test case 32: Xem lịch sử của SV không tồn tại

- Nội dung: Xem lịch sử điểm danh của sinh viên không tồn tại (Mã SV hoặc Tên SV ngoài hệ thống).
- Input: 5 (Lựa chọn menu), SV999 (Mã sinh viên).
- Output mong đợi: “Không tìm thấy sinh viên với mã “SV999”.”.
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 5
Nhập mã sinh viên để xem lịch sử điểm danh: SV999
Không tìm thấy sinh viên với mã 'SV999'.

```

5.6.5 Test case 33: Xem lịch sử của SV, nhập mã lớp không tồn tại

- Nội dung: Xem lịch sử điểm danh của sinh viên nhưng input mã lớp không tồn tại (Mã lớp hoặc Tên lớp ngoài hệ thống).
- Input: 5 (Lựa chọn menu), DATA02 (Mã lớp học).
- Output mong đợi: “Không có bản ghi điểm danh nào phù hợp để hiển thị.”.
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 5
Nhập mã sinh viên để xem lịch sử điểm danh: SV105
Nhập mã lớp để lọc (để trống để xem tất cả các lớp): DATA02
Không có bản ghi điểm danh nào phù hợp để hiển thị.

```

5.7 Kiểm thử chức năng 6: Xem báo cáo điểm danh của lớp học

5.7.1 Test case 34: Xem báo cáo của lớp học, không lọc ngày

- Nội dung:
 - + Trước khi thực hiện test case này, em đã thực hiện test case 1 – điểm danh cho lớp học mã DTB01 – đã được điểm danh trước đó vào một ngày khác.
 - + Xem báo cáo điểm danh của lớp trong toàn bộ các ngày điểm danh.
- Input: 6 (Lựa chọn menu), DTB01 (Mã lớp học), để trống (Ngày điểm danh).
- Output mong đợi: “--- BÁO CÁO ĐIỂM DANH LỚP... ---”.
- Thực hiện và kết quả:

```

--- DANH SÁCH LỚP HỌC ---
- Mã lớp: DTB01, Tên lớp: Database 01, Sĩ số: 22
- Mã lớp: DATA01, Tên lớp: CTDL&GT 01, Sĩ số: 30
- Mã lớp: CODETECH01, Tên lớp: KTLT 01, Sĩ số: 30
- Mã lớp: DTB02, Tên lớp: Database 02, Sĩ số: 0
Nhập mã lớp để xem báo cáo điểm danh: DTB01
Nhập ngày để lọc báo cáo (YYYY-MM-DD, để trống xem tất cả):

```

--- BÁO CÁO ĐIỂM DANH LỚP DATABASE 01 (DTB01) ---					
Ngày	Mã Lớp	Tên Lớp	Mã SV	Tên SV	Trạng thái
2025-06-08	DTB01	Database 01	SV101	Nguyễn Văn A	Vắng
2025-06-08	DTB01	Database 01	SV102	Trần Văn B	Có phép
2025-06-08	DTB01	Database 01	SV104	Hoàng Chí C	Có phép
2025-06-08	DTB01	Database 01	SV105	Đoàn Ngọc D	Vắng
2025-06-08	DTB01	Database 01	SV106	Bùi Đức H	Có mặt
2025-06-08	DTB01	Database 01	SV108	Lê Minh H	Có mặt
2025-06-08	DTB01	Database 01	SV109	Đào Trung H	Có mặt
2025-06-08	DTB01	Database 01	SV112	Hoàng Thái L	Có mặt
2025-06-08	DTB01	Database 01	SV115	Hoàng Bình N	Có mặt
2025-06-08	DTB01	Database 01	SV116	Đỗ Mạnh N	Có mặt
2025-06-08	DTB01	Database 01	SV117	Đào Tiến N	Có mặt
2025-06-08	DTB01	Database 01	SV118	Nguyễn Ngọc O	Có mặt
2025-06-08	DTB01	Database 01	SV119	Dương Thảo Q	Có mặt
2025-06-08	DTB01	Database 01	SV120	Phạm Thị Q	Có mặt
2025-06-08	DTB01	Database 01	SV121	Nông Như T	Có mặt
2025-06-08	DTB01	Database 01	SV122	Phạm Khánh T	Có mặt
2025-06-08	DTB01	Database 01	SV125	Nguyễn Ngọc V	Có mặt
2025-06-08	DTB01	Database 01	SV126	Trần Khánh V	Có mặt
2025-06-08	DTB01	Database 01	SV127	Phạm Xuân V	Có mặt
2025-06-08	DTB01	Database 01	SV128	Kim Duy V	Có mặt
2025-06-08	DTB01	Database 01	SV129	Trần Long V	Có mặt
2025-06-08	DTB01	Database 01	SV130	Vũ Anh X	Có mặt
2025-06-09	DTB01	Database 01	SV101	Nguyễn Văn A	Có mặt
2025-06-09	DTB01	Database 01	SV102	Trần Văn B	Có mặt
2025-06-09	DTB01	Database 01	SV104	Hoàng Chí C	Có mặt
2025-06-09	DTB01	Database 01	SV105	Đoàn Ngọc D	Có mặt
2025-06-09	DTB01	Database 01	SV106	Bùi Đức H	Có mặt
2025-06-09	DTB01	Database 01	SV108	Lê Minh H	Có mặt
2025-06-09	DTB01	Database 01	SV109	Đào Trung H	Có mặt
2025-06-09	DTB01	Database 01	SV112	Hoàng Thái L	Có mặt
2025-06-09	DTB01	Database 01	SV115	Hoàng Bình N	Có mặt
2025-06-09	DTB01	Database 01	SV116	Đỗ Mạnh N	Có mặt
2025-06-09	DTB01	Database 01	SV117	Đào Tiến N	Có mặt
2025-06-09	DTB01	Database 01	SV118	Nguyễn Ngọc O	Có mặt
2025-06-09	DTB01	Database 01	SV119	Dương Thảo Q	Có mặt
2025-06-09	DTB01	Database 01	SV120	Phạm Thị Q	Có mặt
2025-06-09	DTB01	Database 01	SV121	Nông Như T	Có mặt
2025-06-09	DTB01	Database 01	SV122	Phạm Khánh T	Có mặt
2025-06-09	DTB01	Database 01	SV125	Nguyễn Ngọc V	Có mặt
2025-06-09	DTB01	Database 01	SV126	Trần Khánh V	Có mặt
2025-06-09	DTB01	Database 01	SV127	Phạm Xuân V	Có mặt
2025-06-09	DTB01	Database 01	SV128	Kim Duy V	Có mặt
2025-06-09	DTB01	Database 01	SV129	Trần Long V	Có mặt
2025-06-09	DTB01	Database 01	SV130	Vũ Anh X	Có mặt

5.7.2 Test case 35: Xem báo cáo của lớp, có lọc ngày hợp lệ

- Nội dung: Xem báo cáo điểm danh của lớp trong một ngày điểm danh bất kỳ.
- Input: 6 (Lựa chọn menu), DTB01 (Mã lớp học), 2025-06-09 (Ngày điểm danh).
- Output mong đợi: “--- BÁO CÁO ĐIỂM DANH LỚP... – NGÀY ... ---”.
- Thực hiện và kết quả:

```

--- DANH SÁCH LỚP HỌC ---
- Mã lớp: DTB01, Tên lớp: Database 01, Sĩ số: 22
- Mã lớp: DATA01, Tên lớp: CTDL&GT 01, Sĩ số: 30
- Mã lớp: CODETECH01, Tên lớp: KTLT 01, Sĩ số: 30
- Mã lớp: DTB02, Tên lớp: Database 02, Sĩ số: 0
Nhập mã lớp để xem báo cáo điểm danh: DTB01
Nhập ngày để lọc báo cáo (YYYY-MM-DD, để trống xem tất cả): 2025-06-09

```

--- BÁO CÁO ĐIỂM DANH LỚP DATABASE 01 (DTB01) - NGÀY: 2025-06-09 ---					
Ngày	Mã Lớp	Tên Lớp	Mã SV	Tên SV	Trạng thái
2025-06-09	DTB01	Database 01	SV101	Nguyễn Văn A	Có mặt
2025-06-09	DTB01	Database 01	SV102	Trần Văn B	Có mặt
2025-06-09	DTB01	Database 01	SV104	Hoàng Chí C	Có mặt
2025-06-09	DTB01	Database 01	SV105	Đoàn Ngọc D	Có mặt
2025-06-09	DTB01	Database 01	SV106	Bùi Đức H	Có mặt
2025-06-09	DTB01	Database 01	SV108	Lê Minh H	Có mặt
2025-06-09	DTB01	Database 01	SV109	Đào Trung H	Có mặt
2025-06-09	DTB01	Database 01	SV112	Hoàng Thái L	Có mặt
2025-06-09	DTB01	Database 01	SV115	Hoàng Bình N	Có mặt
2025-06-09	DTB01	Database 01	SV116	Đỗ Mạnh N	Có mặt
2025-06-09	DTB01	Database 01	SV117	Đào Tiến N	Có mặt
2025-06-09	DTB01	Database 01	SV118	Nguyễn Ngọc O	Có mặt
2025-06-09	DTB01	Database 01	SV119	Dương Thảo Q	Có mặt
2025-06-09	DTB01	Database 01	SV120	Phạm Thị Q	Có mặt
2025-06-09	DTB01	Database 01	SV121	Nông Như T	Có mặt
2025-06-09	DTB01	Database 01	SV122	Phạm Khánh T	Có mặt
2025-06-09	DTB01	Database 01	SV125	Nguyễn Ngọc V	Có mặt
2025-06-09	DTB01	Database 01	SV126	Trần Khánh V	Có mặt
2025-06-09	DTB01	Database 01	SV127	Phạm Xuân V	Có mặt
2025-06-09	DTB01	Database 01	SV128	Kim Duy V	Có mặt
2025-06-09	DTB01	Database 01	SV129	Trần Long V	Có mặt
2025-06-09	DTB01	Database 01	SV130	Vũ Anh X	Có mặt

5.7.3 Test case 36: Xem báo cáo của lớp, lọc ngày không có dữ liệu

- Nội dung: Xem báo cáo điểm danh của lớp trong một ngày điểm danh vốn không có dữ liệu (chưa điểm danh hôm đó).
- Input: 6 (Lựa chọn menu), DTB01 (Mã lớp học), 2025-06-10 (Ngày điểm danh).
- Output mong đợi: “Không có bản ghi điểm danh nào phù hợp để hiển thị.”.
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 6

Danh sách các lớp học hiện có:

--- DANH SÁCH LỚP HỌC ---
- Mã lớp: DTB01, Tên lớp: Database 01, Sĩ số: 22
- Mã lớp: DATA01, Tên lớp: CTDL&GT 01, Sĩ số: 30
- Mã lớp: CODETECH01, Tên lớp: KTLT 01, Sĩ số: 30
- Mã lớp: DTB02, Tên lớp: Database 02, Sĩ số: 0
Nhập mã lớp để xem báo cáo điểm danh: DTB01
Nhập ngày để lọc báo cáo (YYYY-MM-DD, để trống xem tất cả): 2025-06-10
Không có bản ghi điểm danh nào phù hợp để hiển thị.

```

5.7.4 Test case 37: Xem báo cáo của lớp, lọc ngày sai định dạng

- Nội dung: Xem báo cáo điểm danh của lớp trong một ngày điểm danh nhưng nhập sai định dạng ngày.
- Input: 6 (Lựa chọn menu), DTB01 (Mã lớp học), 09-06-2025 (Ngày điểm danh).
- Output mong đợi: “Định dạng ngày không hợp lệ. Sẽ hiển thị tất cả các ngày cho lớp này.”.
- Thực hiện và kết quả:

```
--- DANH SÁCH LỚP HỌC ---
- Mã lớp: DTB01, Tên lớp: Database 01, Sĩ số: 22
- Mã lớp: DATA01, Tên lớp: CTDL&GT 01, Sĩ số: 30
- Mã lớp: CODETECH01, Tên lớp: KTLT 01, Sĩ số: 30
- Mã lớp: DTB02, Tên lớp: Database 02, Sĩ số: 0
Nhập mã lớp để xem báo cáo điểm danh: DTB01
Nhập ngày để lọc báo cáo (YYYY-MM-DD, để trống xem tất cả): 09-06-2025
Định dạng ngày không hợp lệ. Sẽ hiển thị tất cả các ngày cho lớp này.
```

--- BÁO CÁO ĐIỂM DANH LỚP DATABASE 01 (DTB01) ---					
Ngày	Mã Lớp	Tên Lớp	Mã SV	Tên SV	Trạng thái
2025-06-08	DTB01	Database 01	SV101	Nguyễn Văn A	Vắng
2025-06-08	DTB01	Database 01	SV102	Trần Văn B	Có phép
2025-06-08	DTB01	Database 01	SV104	Hoàng Chí C	Có phép
2025-06-08	DTB01	Database 01	SV105	Đoàn Ngọc D	Vắng
2025-06-08	DTB01	Database 01	SV106	Bùi Đức H	Có mặt
2025-06-08	DTB01	Database 01	SV108	Lê Minh H	Có mặt
2025-06-08	DTB01	Database 01	SV109	Đào Trung H	Có mặt
2025-06-08	DTB01	Database 01	SV112	Hoàng Thái L	Có mặt
2025-06-08	DTB01	Database 01	SV115	Hoàng Bình N	Có mặt
2025-06-08	DTB01	Database 01	SV116	Đỗ Mạnh N	Có mặt
2025-06-08	DTB01	Database 01	SV117	Đào Tiến N	Có mặt
2025-06-08	DTB01	Database 01	SV118	Nguyễn Ngọc O	Có mặt
2025-06-08	DTB01	Database 01	SV119	Dương Thảo Q	Có mặt
2025-06-08	DTB01	Database 01	SV120	Phạm Thị Q	Có mặt
2025-06-08	DTB01	Database 01	SV121	Nông Như T	Có mặt
2025-06-08	DTB01	Database 01	SV122	Phạm Khánh T	Có mặt
2025-06-08	DTB01	Database 01	SV125	Nguyễn Ngọc V	Có mặt
2025-06-08	DTB01	Database 01	SV126	Trần Khánh V	Có mặt
2025-06-08	DTB01	Database 01	SV127	Phạm Xuân V	Có mặt
2025-06-08	DTB01	Database 01	SV128	Kim Duy V	Có mặt
2025-06-08	DTB01	Database 01	SV129	Trần Long V	Có mặt
2025-06-08	DTB01	Database 01	SV130	Vũ Anh X	Có mặt
2025-06-09	DTB01	Database 01	SV101	Nguyễn Văn A	Có mặt
2025-06-09	DTB01	Database 01	SV102	Trần Văn B	Có mặt
2025-06-09	DTB01	Database 01	SV104	Hoàng Chí C	Có mặt
2025-06-09	DTB01	Database 01	SV105	Đoàn Ngọc D	Có mặt
2025-06-09	DTB01	Database 01	SV106	Bùi Đức H	Có mặt
2025-06-09	DTB01	Database 01	SV108	Lê Minh H	Có mặt
2025-06-09	DTB01	Database 01	SV109	Đào Trung H	Có mặt
2025-06-09	DTB01	Database 01	SV112	Hoàng Thái L	Có mặt
2025-06-09	DTB01	Database 01	SV115	Hoàng Bình N	Có mặt
2025-06-09	DTB01	Database 01	SV116	Đỗ Mạnh N	Có mặt
2025-06-09	DTB01	Database 01	SV117	Đào Tiến N	Có mặt
2025-06-09	DTB01	Database 01	SV118	Nguyễn Ngọc O	Có mặt
2025-06-09	DTB01	Database 01	SV119	Dương Thảo Q	Có mặt
2025-06-09	DTB01	Database 01	SV120	Phạm Thị Q	Có mặt
2025-06-09	DTB01	Database 01	SV121	Nông Như T	Có mặt
2025-06-09	DTB01	Database 01	SV122	Phạm Khánh T	Có mặt
2025-06-09	DTB01	Database 01	SV125	Nguyễn Ngọc V	Có mặt
2025-06-09	DTB01	Database 01	SV126	Trần Khánh V	Có mặt
2025-06-09	DTB01	Database 01	SV127	Phạm Xuân V	Có mặt
2025-06-09	DTB01	Database 01	SV128	Kim Duy V	Có mặt
2025-06-09	DTB01	Database 01	SV129	Trần Long V	Có mặt
2025-06-09	DTB01	Database 01	SV130	Vũ Anh X	Có mặt

5.7.5 Test case 38: Xem báo cáo cho lớp không tồn tại

- Nội dung: Xem báo cáo điểm danh của của một lớp vốn không có trong danh sách đầu vào lớp học.
- Input: 6 (Lựa chọn menu), DTB04 (Mã lớp học).

- Output mong đợi: “Lớp học với mã “DTB04” không tồn tại.”.
- Thực hiện và kết quả:

```

--- HỆ THỐNG ĐIỂM DANH (MVC) ---
1. Điểm danh cho lớp học
2. Cập nhật trạng thái điểm danh
3. Tìm kiếm sinh viên
4. Tìm kiếm lớp học
5. Xem lịch sử điểm danh của sinh viên
6. Xem báo cáo điểm danh của lớp
0. Thoát
Nhập lựa chọn của bạn: 6

Danh sách các lớp học hiện có:

--- DANH SÁCH LỚP HỌC ---
- Mã lớp: DTB01, Tên lớp: Database 01, Sĩ số: 22
- Mã lớp: DATA01, Tên lớp: CTDL&GT 01, Sĩ số: 30
- Mã lớp: CODETECH01, Tên lớp: KTLT 01, Sĩ số: 30
- Mã lớp: DTB02, Tên lớp: Database 02, Sĩ số: 0
Nhập mã lớp để xem báo cáo điểm danh: DTB04
Lớp học với mã 'DTB04' không tồn tại.

```

CHƯƠNG 6. TỔNG KẾT

6.1 Ưu điểm

Về ưu điểm, chương trình của em đã thực hiện đầy đủ các chức năng so với yêu cầu của bài tập lớn đã giao là thiết kế hệ thống cho phép ghi nhận điểm danh theo ngày cho từng sinh viên (mã sinh viên, họ tên) cho các lớp học và cho phép tìm kiếm theo ngày của lớp hoặc theo mã sinh viên.

Thiết kế chương trình bảo đảm được nhiều tiêu chí cấu trúc tốt như: Thiết kế theo cơ chế top – down, module hóa; tách biệt trách nhiệm giữa các module và thỏa mãn các đặc tính của một chương trình tốt như:

- Tính nhất quán: Việc đặt tên các thành phần trong chương trình có sự nhất quán cao (Các thành phần (biến cục bộ, biến toàn cục, hàm,...đều tuân thủ theo các quy tắc đặt tên nhất định).
- Tính bao đóng: Đóng gói dữ liệu (thuộc tính) và phương thức trong các lớp đối tượng, các hàm trong chương trình thực hiện một nhiệm vụ cụ thể và chỉ tác động đến giá trị trả về.
- Tính rõ ràng: Viết docstring đầy đủ ở đầu mỗi module, đầu các hàm và có chú thích rõ ràng cho các phần trong chương trình; việc đặt tên hàm, tên biến, tên module đều thể hiện rõ vai trò, công việc của chúng.

Bên cạnh đó, Việc kết hợp debug và kiểm thử hồi quy đã giúp chương trình dần trở nên hoàn thiện dần qua từng lần kiểm tra, ổn định hơn qua từng phiên bản.

6.2 Nhược điểm và hướng mở rộng

Chương trình chưa có giao diện đồ họa (GUI) mà chỉ có giao diện dòng lệnh, tương đối không thân thiện với người dùng bình thường. Hướng phát triển đó là xây dựng giao diện đồ họa khiến các thao tác trở nên dễ quan sát hơn với người dùng và đáp ứng mạnh mẽ các tiêu chí của mô thức lập trình hướng sự kiện.

Việc sử dụng biến toàn cục làm cache có thể gây khó khăn nếu quy mô hệ thống được mở rộng và tồn tại nhiều tiến trình/luồng hơn. Giải pháp khắc phục có thể kể ra là sử dụng một class chuyên dùng để caching dữ liệu.

Cần một bộ dữ liệu (file unit test) để có thể kiểm thử tự động thay vì phải chạy từng test case một cách thủ công, sử dụng hai thư viện unittest và coverage trong Python.

Việc thêm/xóa sinh viên, lớp học trong dữ liệu đầu vào phải thực hiện thủ công trong các file JSON, có thể gây bất tiện cho người dùng bình thường. Hướng mở rộng là phát triển hai chức năng mới cho phép thêm và xóa sinh viên, lớp học; đảm bảo hệ thống vừa là quản lý sinh viên, quản lý lớp học kiêm quản lý điểm danh.

Chuyển đổi sang dùng cơ sở dữ liệu thực thụ là một hướng phát triển tốt, thay thế cho việc dùng file JSON.

CHƯƠNG 7. PHỤ LỤC

7.1 Mã nguồn hàm main

```
"""CHƯƠNG TRÌNH CHÍNH"""

from program_controller import AppController # Import lớp Controller

if __name__ == "__main__":
    app_controller = AppController() # Khởi tạo Controller

    # Vòng lặp chính của chương trình
    running = True
    while running:
        choice = app_controller.display_main_menu()
        running = app_controller.handle_choices(choice)
        if running and choice != '0':
            input("\nNhấn Enter để tiếp tục...")
```

7.2 Đặc tả các chương trình con

Với từng chương trình con trong các file module, em đã đặc tả ở đầu mỗi hàm/phương thức bên trong docstring với cấu trúc đặc tả bao gồm:

- Đầu vào của hàm/phương thức.
- Đầu ra của hàm/phương thức.
- Mô tả chức năng của hàm/phương thức.

Em sẽ trình bày một số ví dụ về đặc tả chương trình con trong các file module (mỗi module em chỉ lấy ví dụ một hàm được đặc tả). Nếu thầy muốn xem chi tiết hơn thì có thể tham khảo mã nguồn

- Đặc tả trong module 1 – models.py (Ví dụ):

```
def __str__(self) -> str:
    """
    Đầu vào: self - tham chiếu đến đối tượng (ngầm định).
    Đầu ra: Không có
    Phương thức này trả về chuỗi đại diện cho đối tượng Student
    thân thiện với người dùng"""
    return f"MSSV: {self.student_id}, Tên: {self.full_name}"
```

- Đặc tả trong module 2 – data_manager.py (Ví dụ):

```
def _save_data_to_file(filepath: str, data: List[Dict[str, Any]]):
    """
    Đầu vào:
    - filepath (kiểu str): Đường dẫn đầy đủ đến file JSON để ghi dữ liệu.
    - data (kiểu List[Dict[str, Any]]): Danh sách các dict cần ghi vào file.
    Đầu ra: Không có (None).
    Hàm này ghi một danh sách các dict vào file JSON được chỉ định.
    """
    try:
        with open(filepath, 'w', encoding='utf-8') as f:
            json.dump(data, f, indent=4, ensure_ascii=False)
    except IOError:
        print(f"Lỗi IOError khi ghi file: {filepath}")
    except Exception as e:
        print(f"Lỗi không xác định khi ghi file {filepath}: {e}")
```

- Đặc tả trong module 3 – attendance_taking_system.py (Ví dụ):


```
def get_student_by_id(student_id: str) -> Optional[Student]:
    """
    Đầu vào: student_id (kiểu str): Mã số sinh viên cần tìm.
    Đầu ra: Một đối tượng Student nếu tìm thấy, ngược lại trả về None
    Hàm này tìm kiếm và trả về một đối tượng Student
    | từ cache dựa trên mã sinh viên được cung cấp
    """
    load_all_data_once()
    for student in ALL_STUDENTS:
        if student.student_id == student_id: # So sánh chính xác
            return student
    return None
```

- Đặc tả trong module 4 – search_service.py (Ví dụ):

```
def search_class_by_name_or_id(query: str) -> List[Class]:
    """
    Đầu vào: query (kiểu str): Chuỗi tìm kiếm (Mã lớp/Tên lớp/1 phần tên lớp)
    Đầu ra: Một danh sách các đối tượng Class (kiểu List[Class]) phù hợp.
    Hàm này tìm kiếm lớp học trong bộ đệm. Nếu tìm bằng mã lớp thì
    | yêu cầu nhập chính xác, nếu tìm bằng tên lớp thì không cần phải nhập
    | chính xác chữ hoa/chữ thường.
    """
    _ensure_data_loaded() # Đảm bảo dữ liệu sẵn sàng
    results = [] # Tạo danh sách lưu kết quả

    # Chuẩn hóa chuỗi tìm kiếm được nhập
    query_stripped = query.strip()
    if not query_stripped:
        return []

    # Tìm kiếm lớp học phù hợp với đầu vào tìm kiếm
    for school_class_item in attendance_taking_system.ALL_CLASSES:
        if query_stripped == school_class_item.class_id or \
            query_stripped.lower() in school_class_item.class_name.lower():
            results.append(school_class_item)
    return results
```

- Đặc tả trong module 5 – display_search_result.py (Ví dụ):

```
def display_found_students(students: List[Student]):
    """
    Đầu vào: students (kiểu List[Student]): Một DS các đối tượng Student.
    Đầu ra: Không có.
    Hàm này hiển thị thông tin sinh viên trong danh sách được cung cấp.
    """
    if not students: # Nếu không tìm thấy SV phù hợp
        print("Không tìm thấy sinh viên nào phù hợp.")
        return
    print("\n--- SINH VIÊN TÌM THẤY ---")
    for s in students:
        print(f"    - Mã SV: {s.student_id}, Tên: {s.full_name}")
```

- Đặc tả trong module 6 – program_controller.py (Ví dụ):

```
def display_main_menu(self):
    """
    Đầu vào: Self - tham chiếu đối tượng (ngầm định).
    Đầu ra: Một chuỗi (str) là lựa chọn do người dùng nhập.
    Phương thức này hiển thị menu các lựa chọn cho người dùng
    và nhận đầu vào là lựa chọn.
    """

    print("\n--- HỆ THỐNG ĐIỂM DANH (MVC) ---")
    print("1. Điểm danh cho lớp học")
    print("2. Cập nhật trạng thái điểm danh")
    print("3. Tìm kiếm sinh viên")
    print("4. Tìm kiếm lớp học")
    print("5. Xem lịch sử điểm danh của sinh viên")
    print("6. Xem báo cáo điểm danh của lớp")
    print("0. Thoát")
    return input("Nhập lựa chọn của bạn: ")
```

TÀI LIỆU THAM KHẢO

- [1] Slide bài giảng của thầy Vũ Thành Nam.
- [2] <https://www.w3schools.com/python/>
- [3] <https://github.com/topics/attendance-management-system>
- [4] https://github.com/Spidy20/Attendace_management_system
- [5] <https://gemini.google.com/app>
- [6] <https://stackoverflow.com/questions>